Árvores

Estrutura de Dados

Universidade Federal de Juiz de Fora
Departamento de Ciência da Computação

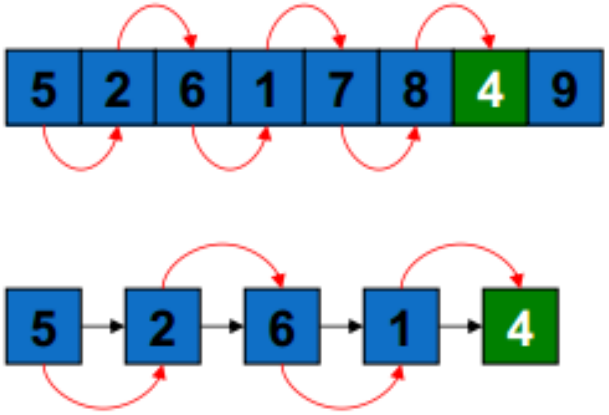
Conteúdo

- Introdução
- Árvores
  - Definições
  - Conceitos
  - Algoritmos
- Árvore Binária
  - Implementação
  - Algoritmos
- Árvore Binária de Busca
  - Implementação
  - Algoritmos
- Exercícios

Introdução

Pesquisa sequencial

- Problema: determinar se um valor existe ou não em um vetor.
- Os elementos são pesquisados sucessivamente.
- Comparação determina se o elemento foi encontrado ou não.
- Exemplo: buscar 4 (vetor ou lista encadeada).



- Tempo de busca: $O(n)$.

Pesquisa sequencial

- Se a pesquisa sequencial é executada em um vetor **não ordenado**, então é necessário pesquisar em todo o vetor:

```
int busca(int v[], int tam, int chave)
{
  int i = 0;

  while (chave != v[i] && i < tam)
    i = i + 1;

  if (chave == v[i])
    return i;
  else
    return -1;
}
```

Pesquisa sequencial

- Pesquisa sequencial executada em um vetor **ordenado**:

```
int busca(int v[], int tam, int chave)
{
  int i = 0;

  while (chave > v[i] && i < tam)
    i = i + 1;

  if (chave == v[i])
    return i;
  else
    return -1;
}
```

Pesquisa binária

- Pesquisa binária em vetor **ordenado (crescente)**:

```
int buscaBin(int v[], int tam, int chave)
{
  int esq = 0, dir = tam - 1;
  int pos = (esq + dir) / 2;
  while (chave != v[pos] && esq <= dir)  {
    if (chave < v[pos])
      dir = pos - 1;
    else
      esq = pos + 1;
    pos = (esq + dir) / 2;
  }
  if (chave == v[pos])
    return pos;
  else
    return -1;
}
```

- Tempo de busca: $O(\log n)$

Pesquisa binária

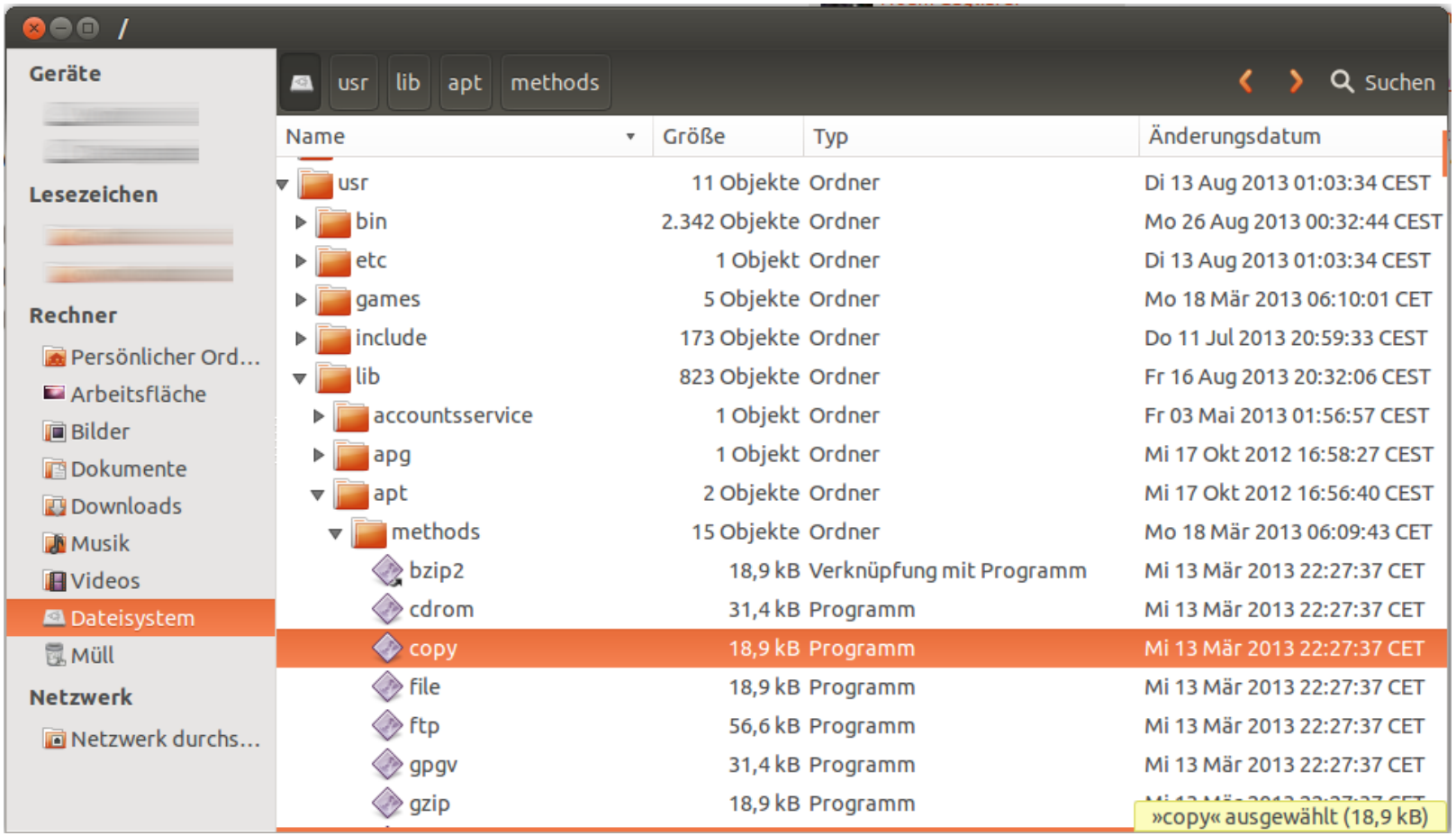
- Implementação **recursiva** da pesquisa binária:

```
int buscaBin(int v[], int imin, int imax,
             int chave)
{
  if (imax < imin)
    return -1;
  else
  {
    int imed = (imin+max)/2;

    if (v[imed] > chave)
      return buscaBin(A, imin, imed-1, chave);
    else if (v[imed] < chave)
      return buscaBin(A, imed+1, imax, chave);
    else
      return imed;
  }
}
```

Árvores

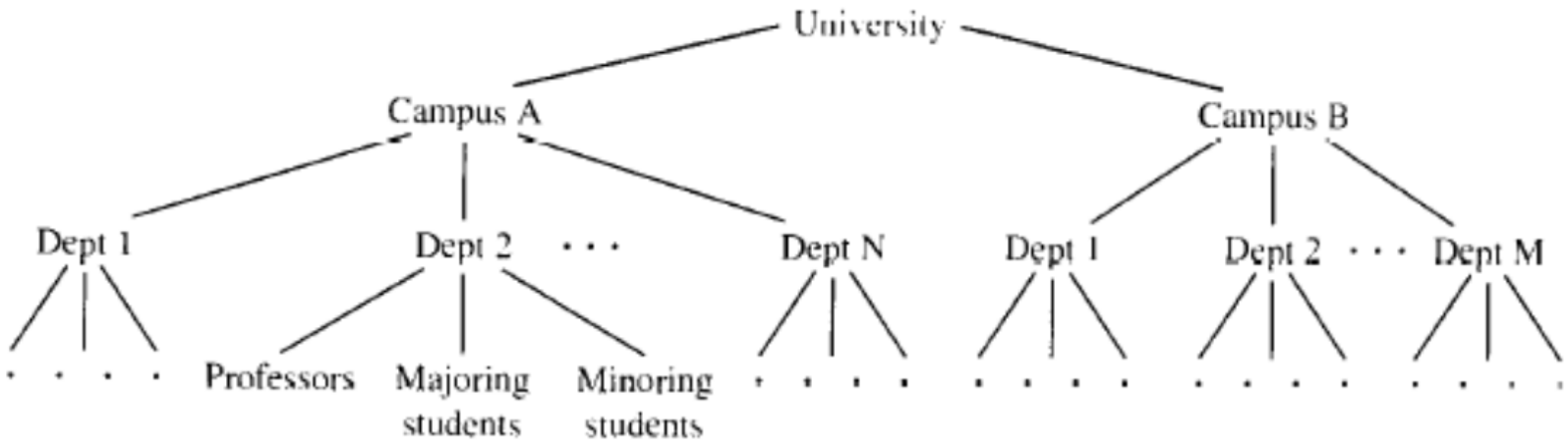
Motivação

- Qual estrutura de dados um gerenciador de arquivos deve utilizar?



Motivação

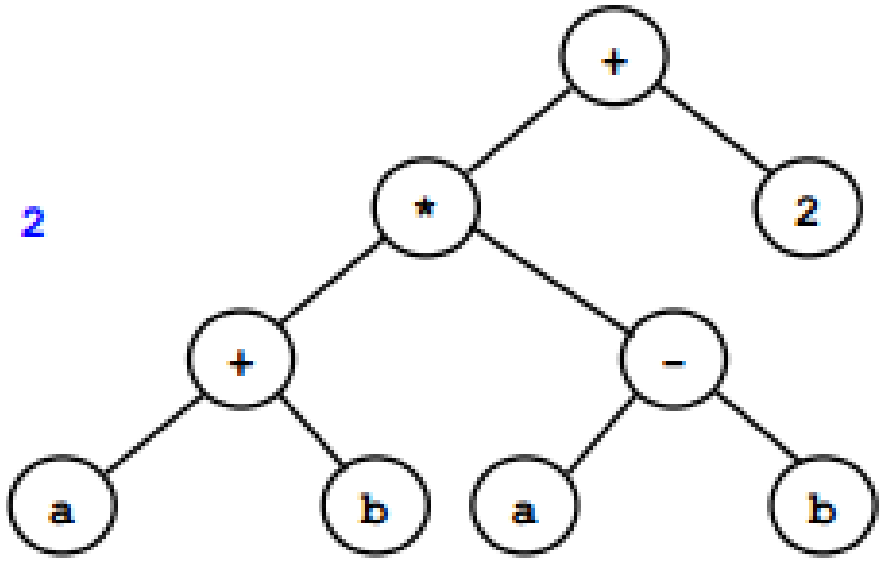
- Hierarquia (ex: universidade)



Motivação

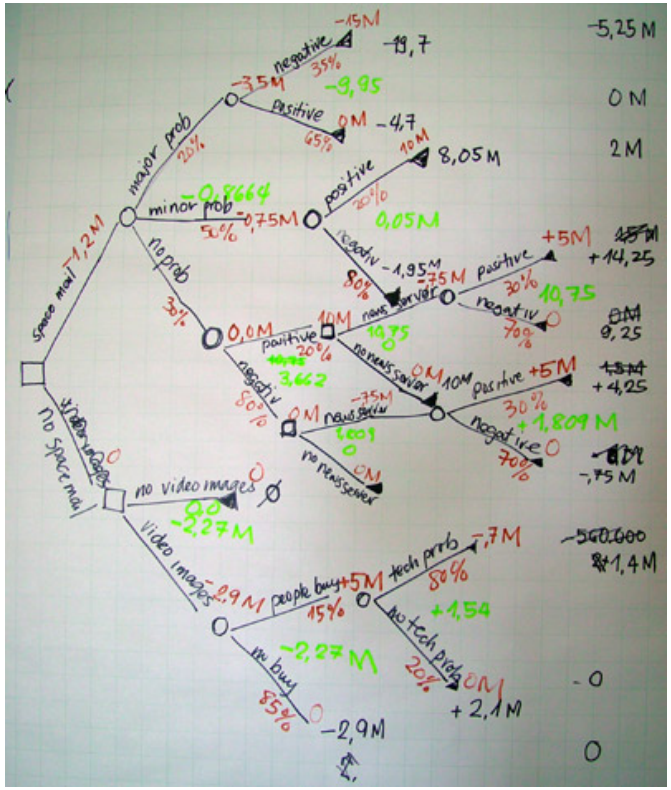
- Representação computacional de expressões aritméticas

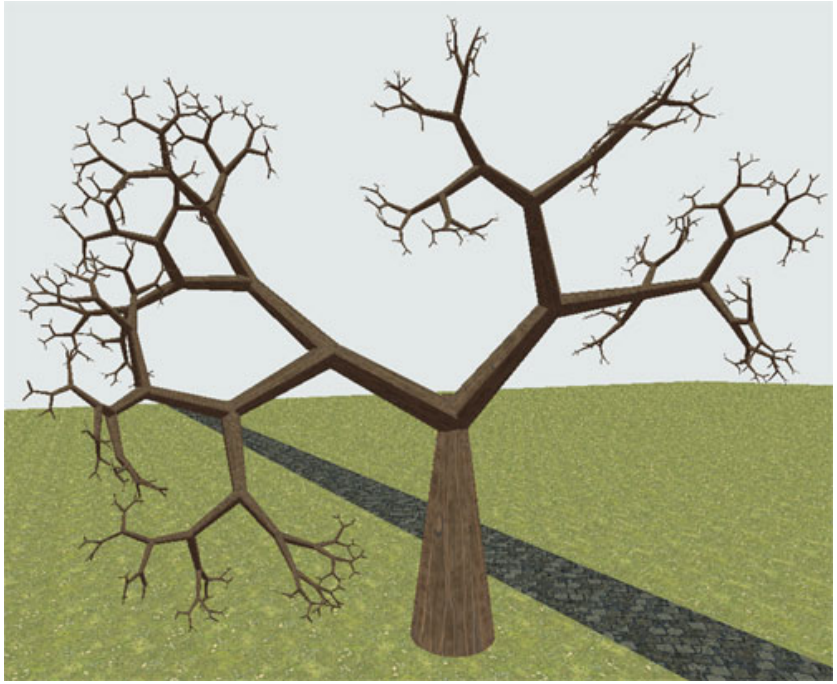
(a+b) * (a-b) + 2



Motivação

- E muitas outras aplicações...



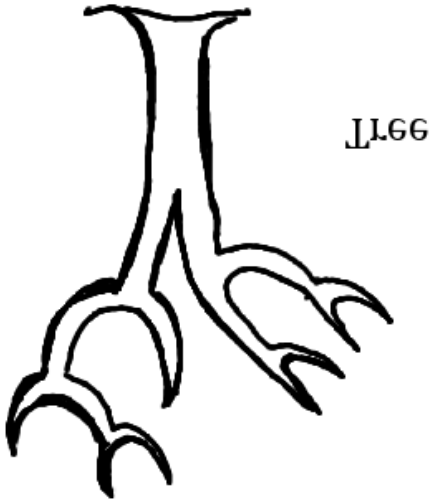


Motivação

- Listas ligadas
  - São mais flexíveis do que vetores;
  - Possuem acesso sequencial;
  - São estruturas lineares sendo difícil utilizá-las para organizar representação hierárquica de objetos.
- Pilhas e filas
  - Refletem alguma hierarquia;
  - Mas são limitadas a somente uma dimensão.

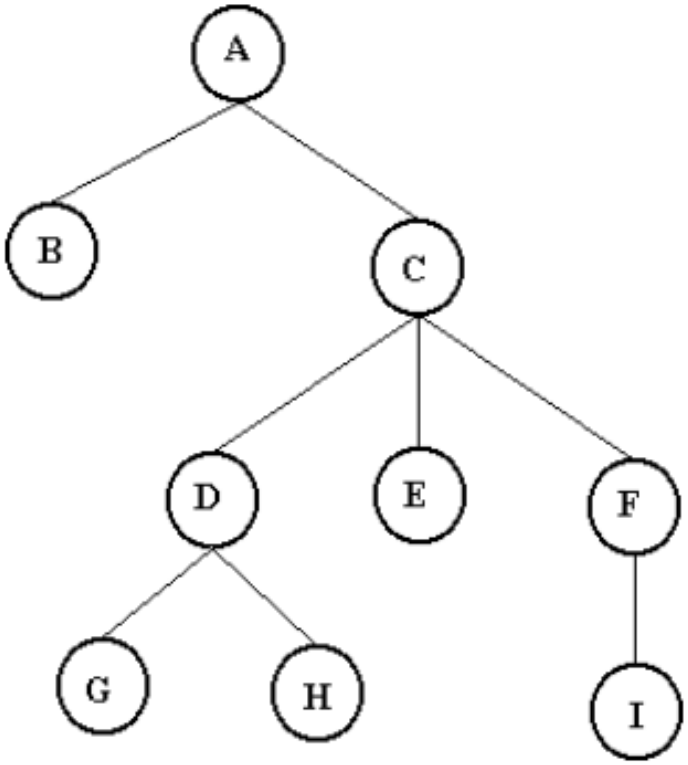
Árvores

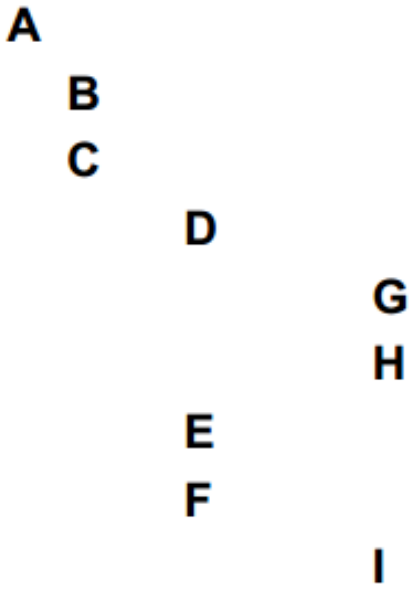
- **Árvores** são estruturas criadas para superar as limitações de listas ligadas, pilhas e filas.
- Consiste de nós e de arcos.
- São representadas com a raiz no topo e as folhas na base (diferente de árvore natural).



Representações Gráficas de Árvores

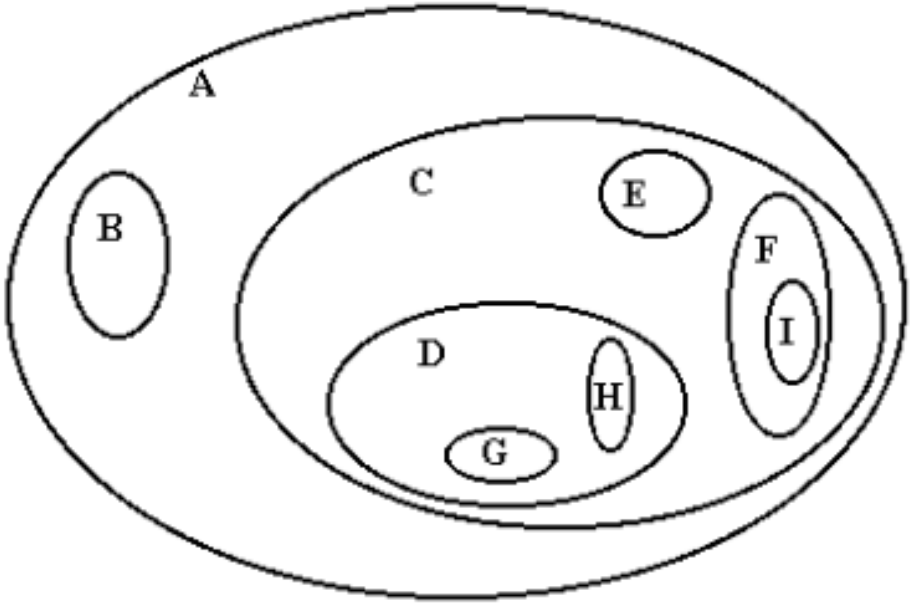
- Hierárquica ou alinhamento dos nós.





Representações Gráficas de Árvores

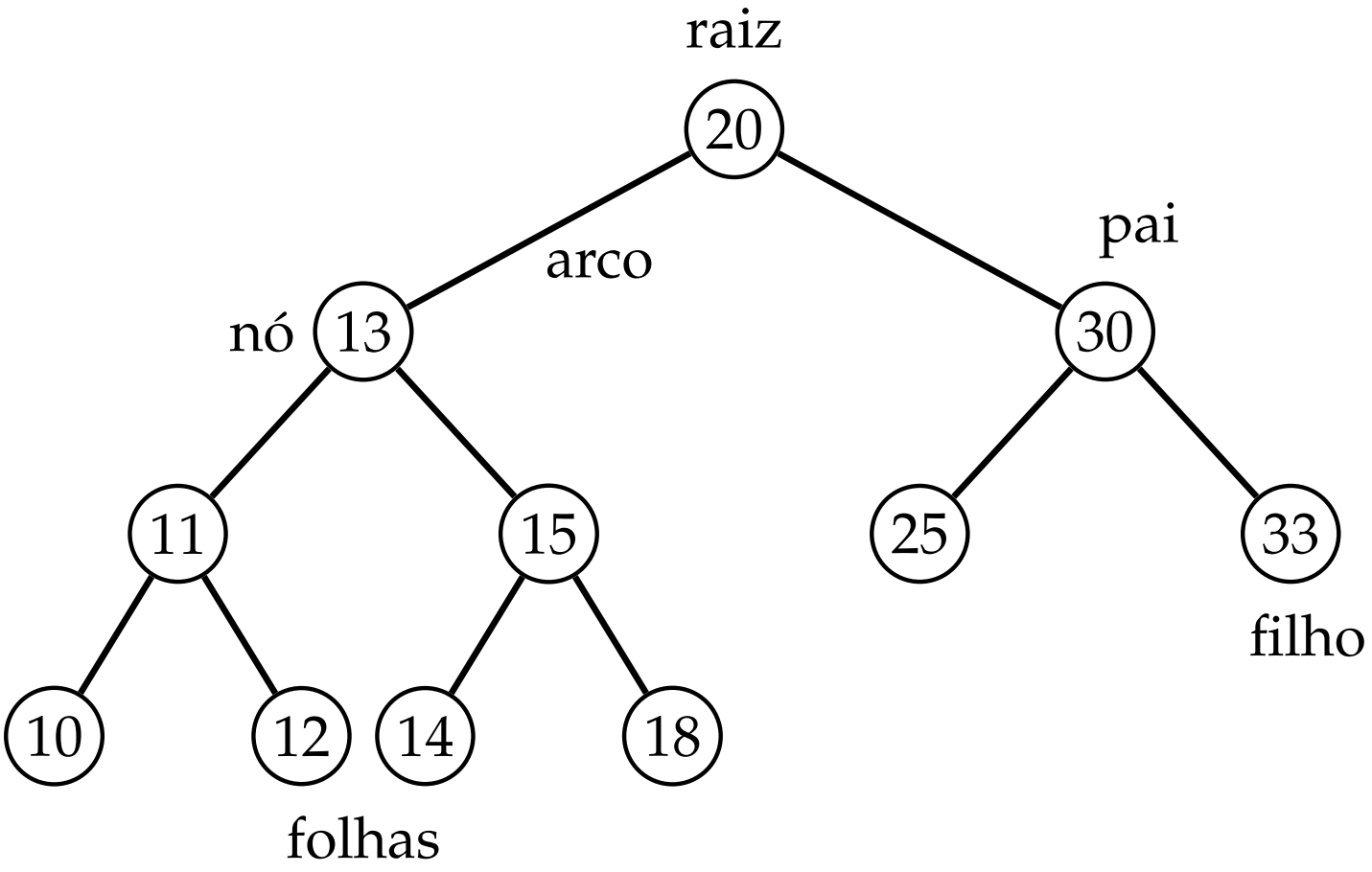
- Parênteses aninhados: (A (B) (C (D (G) (H)) (E) (F (I))))
- Diagramas de inclusão:



Definições

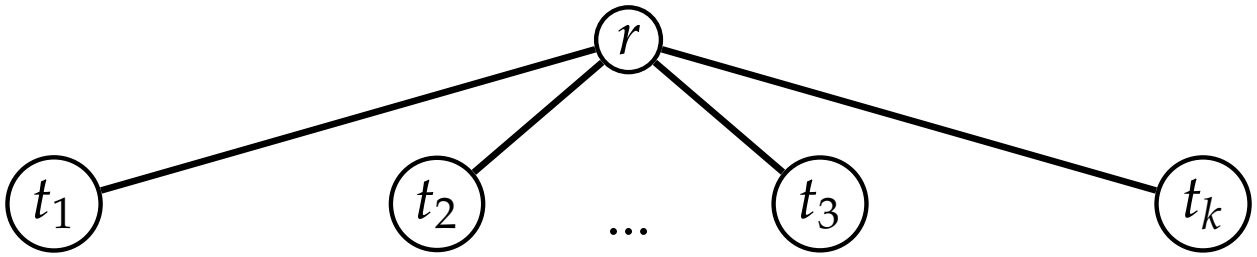
- Nó: Elemento que contém a informação
- Arco: Liga dois nós
- Pai: nó superior de um arco
- Filho: nó inferior de um arco
- Raiz (nó topo)
  - Não possui ancestrais (não tem nó pai)
  - Só pode ter filhos
- Folhas
  - Nós das extremidades inferiores
  - Não têm nós filhos (ou melhor, seus filhos são estruturas vazias)

Definições



Definições

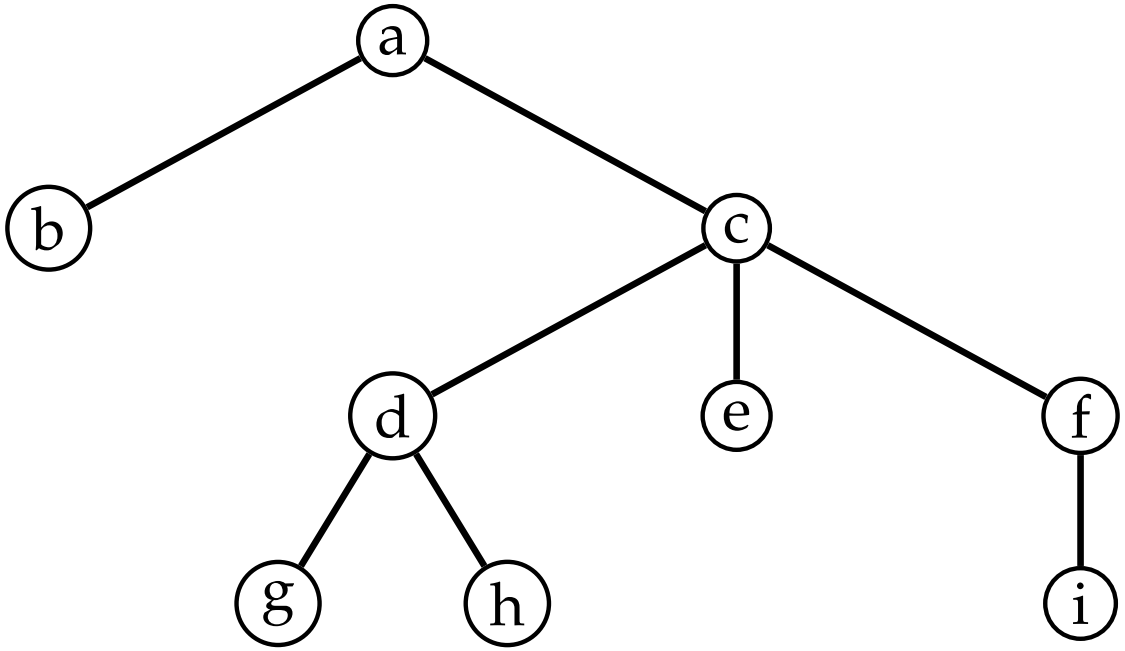
- Definição **recursiva** de árvore
  1. Uma estrutura vazia é uma árvore vazia.
  2. Se $t_1 \ldots, t_k$ são raízes de árvores disjuntas, então a estrutura cuja raiz r tem como suas filhas as raízes $t_1 \ldots, t_k$ também é uma árvore.
  3. Somente estruturas geradas pelas regras 1 e 2 são árvores.



Definições

- **Grau de um nó** é o número de subárvores de um nó.
- **Grau de uma árvore** (aridade): é definido como sendo igual ao grau máximo entre todos os nós da árvore.
- Cada nó tem que ser atingível a partir da raiz, através de uma sequência única de arcos chamada de **caminho**.
- O **comprimento** do caminho é o número de arcos do caminho.
- **Nível** de um nó é a distância (em arcos) entre o nó e a raiz da árvore. A raiz tem nível igual a 0.

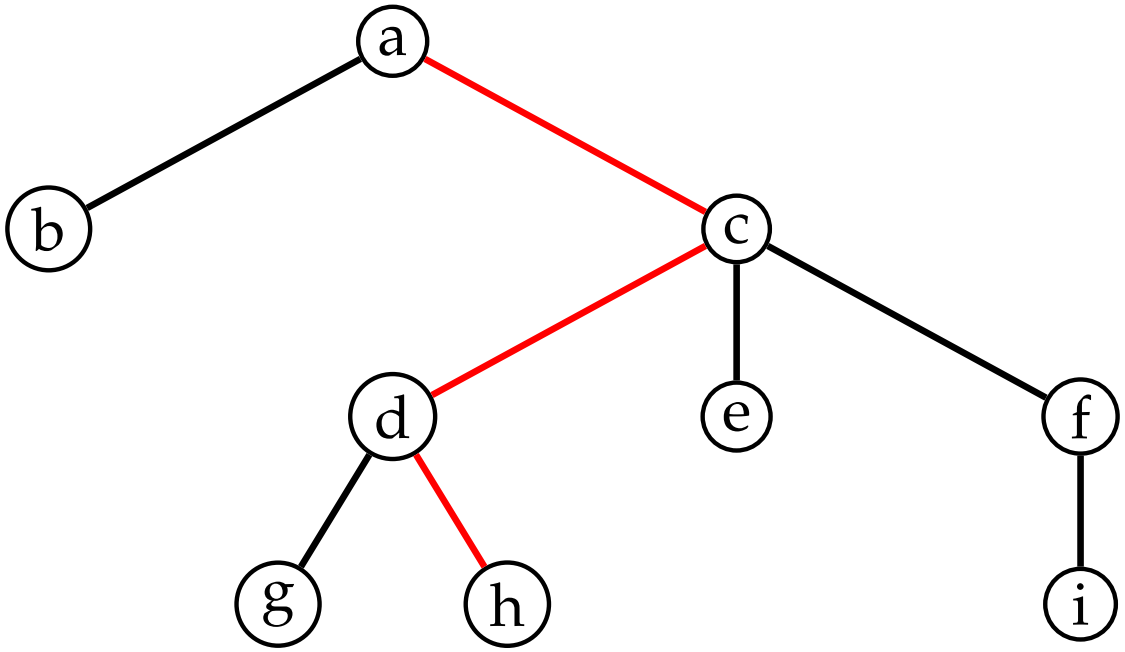
Exemplo



- Grau dos nós:
  - G(a)=2, G(b)=0, G(c)=3, G(d)=2,
  - G(e)=0, G(f)=1, G(g)=0, G(h)=0, G(i)=0.
- Grau da árvore: G(T)=3

Exemplo

- Exemplo do caminho que vai do nó *a* até o nó *h*.
- O comprimento desse caminho é 3.
- Nesse exemplo, o nó *a* tem nível 0; os nós *b* e *c* têm nível 1; o nó *h* tem nível 3.

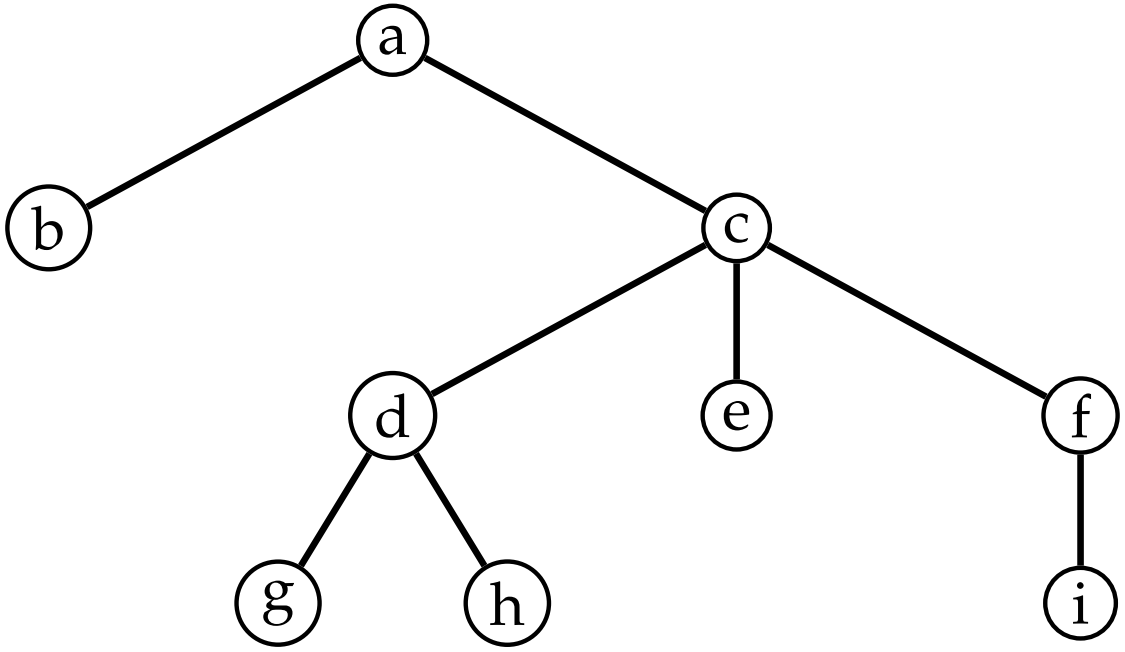


Definições

- **Altura de um nó** x é a distância entre x e o seu descendente mais afastado (folha). Mais precisamente, a altura de x é o número de arcos do mais longo caminho que vai de x até uma folha.
- **Altura da árvore** é o nível do nó folha que tem o mais longo caminho até a raiz.
  - A árvore vazia é uma árvore de altura -1, por definição.
  - Uma árvore com um único nó tem altura 0.
  - O nó é raiz e folha ao mesmo tempo.
- Toda árvore com $n > 1$ nós possui:
  - no mínimo 1 folha;
  - e no máximo $n - 1$ folhas.

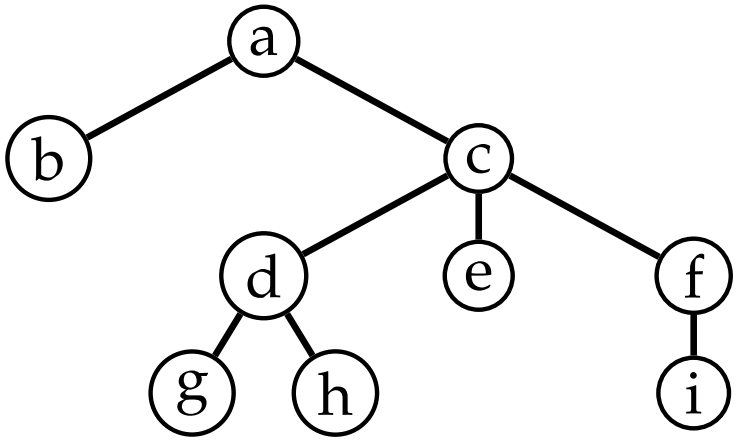
Exemplo

- Quais os níveis e a altura da seguinte árvore?

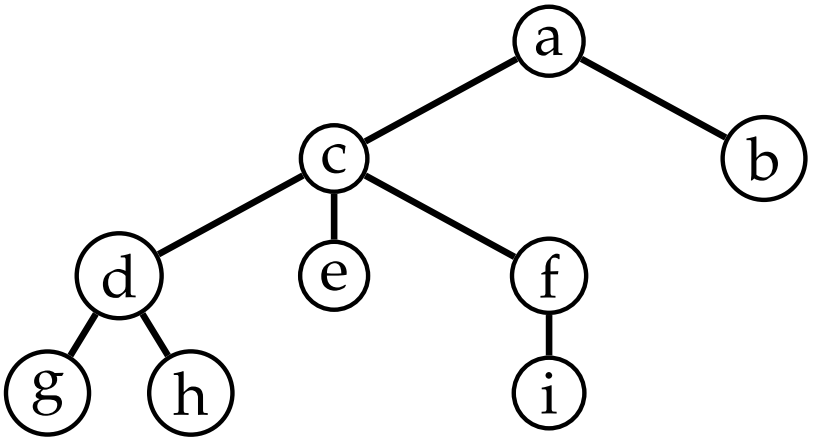


Definições

- **Árvore ordenada**: os filhos de cada nó estão ordenados (assume-se ordenação da esquerda para a direita).
- Exemplos:



Ordenada



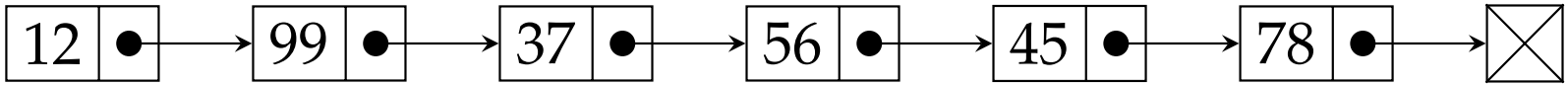
Não-ordenada

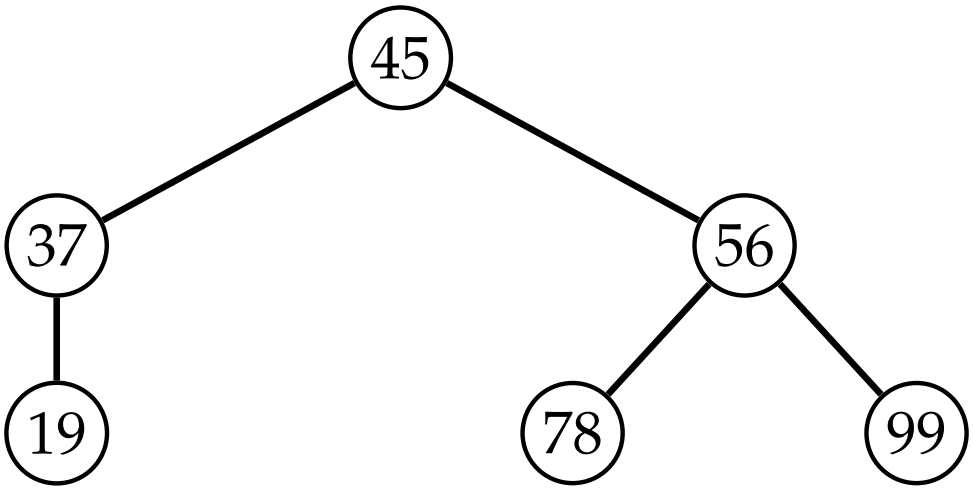
Árvores

- Note que, a definição de árvore não impõe qualquer condição sobre o número de filhos de um nó:
  - Pode variar de 0 a qualquer inteiro;
  - Árvores são muito utilizadas em sistemas gerenciadores de banco de dados.

Árvores

- Considerando uma lista encadeada e uma árvore (abaixo), qual pesquisa é mais rápida para achar um valor (chave)?

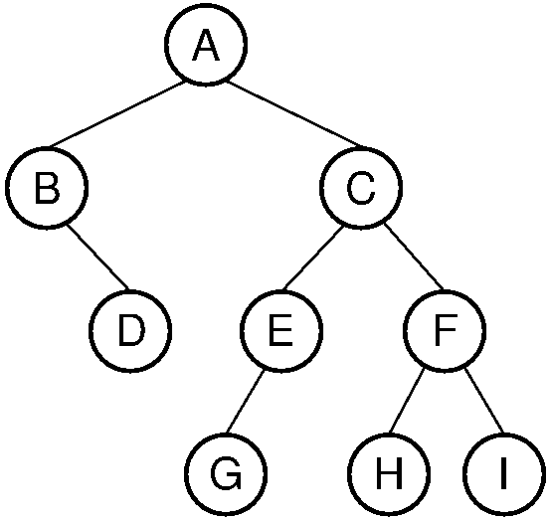


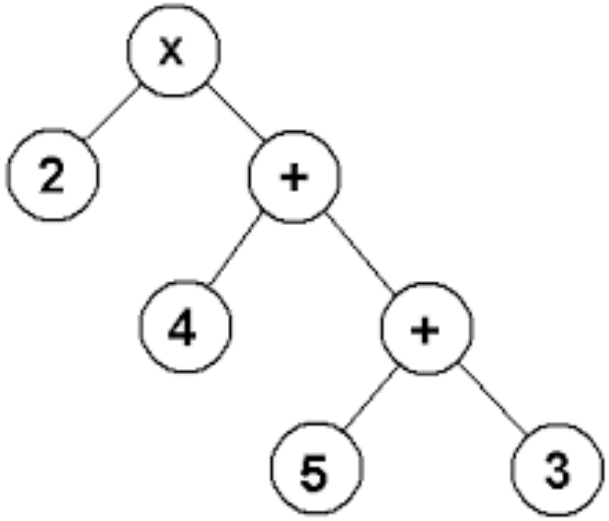


Árvore Binária

Árvores Binárias

- Uma árvore binária é uma árvore cujos nós têm no máximo, dois filhos, portanto, a árvore binária **tem grau 2**.
- Em uma árvore binária cada filho é designado como filho à esquerda ou filho à direita (ou nó filho esquerdo e nó filho direito).
- Exemplos:



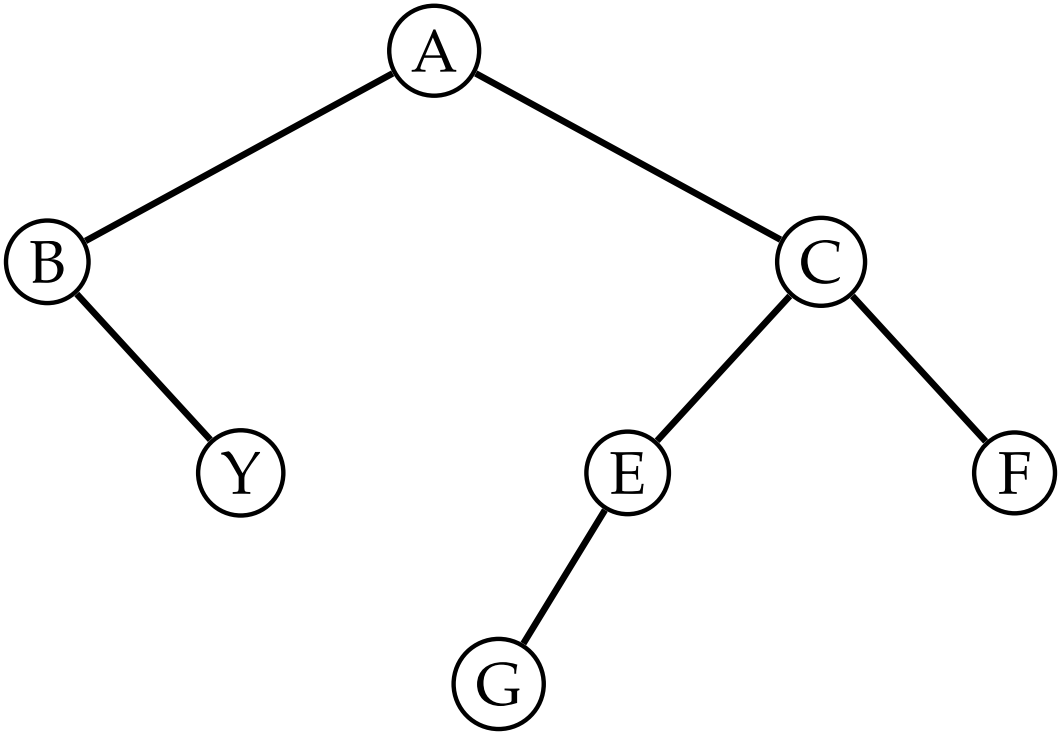


Árvores Binárias

- Definição: Uma árvore binária T é um conjunto finito de elementos denominados nós ou vértices, tal que:
  - se T = $\emptyset$, a árvore é dita vazia, ou
  - existe um nó especial r, chamado raiz de T, e os nós restantes podem ser divididos em dois subconjuntos disjuntos, T_r^L e T_r^R, correspondentes às **subárvores a esquerda e a direita** de r, respectivamente, que também são árvores binárias.

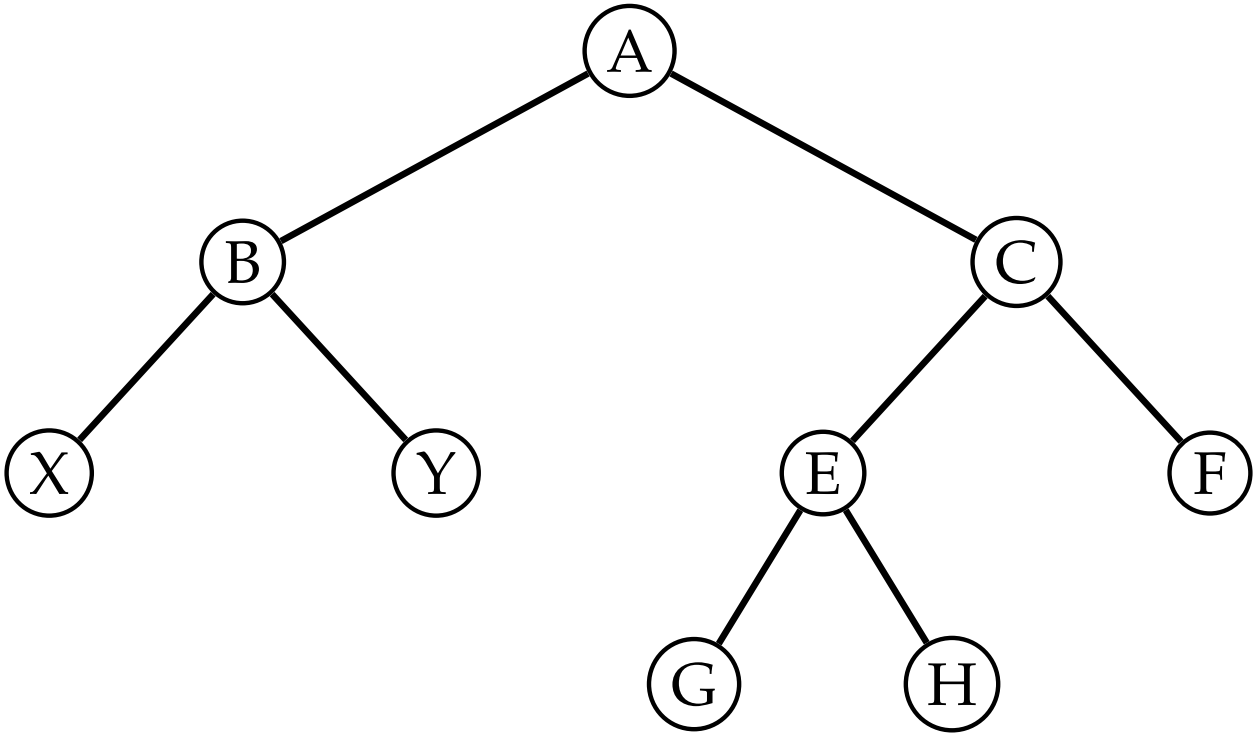
Árvores Binárias

- Exemplo



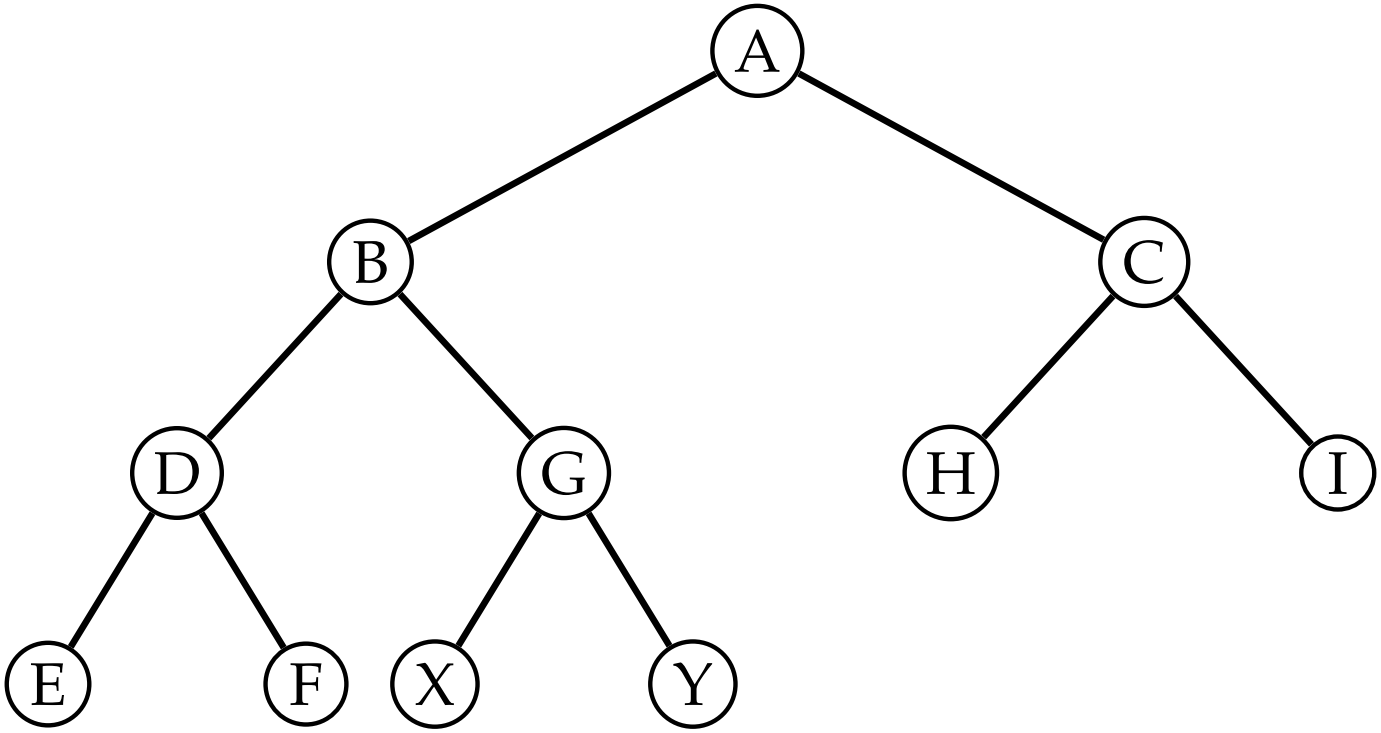
Árvores Binárias

- Árvore **estritamente** binária: cada nó possui 0 ou 2 filhos.



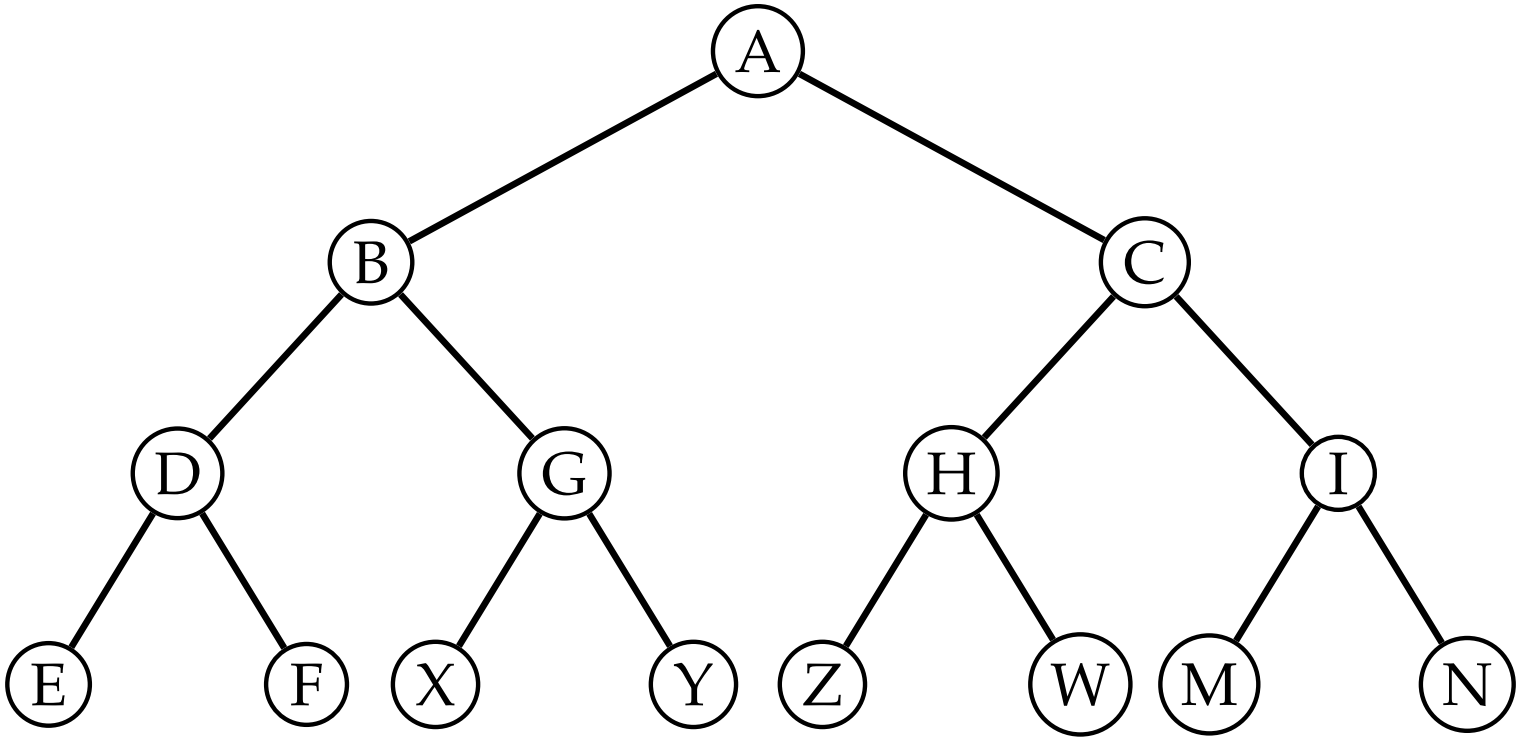
Árvores Binárias

- Árvore binária **completa**: uma árvore binária T com n níveis é completa se todos os níveis, exceto possivelmente o último estão completamente cheios, e o último nível tem todos os seus nós à esquerda.



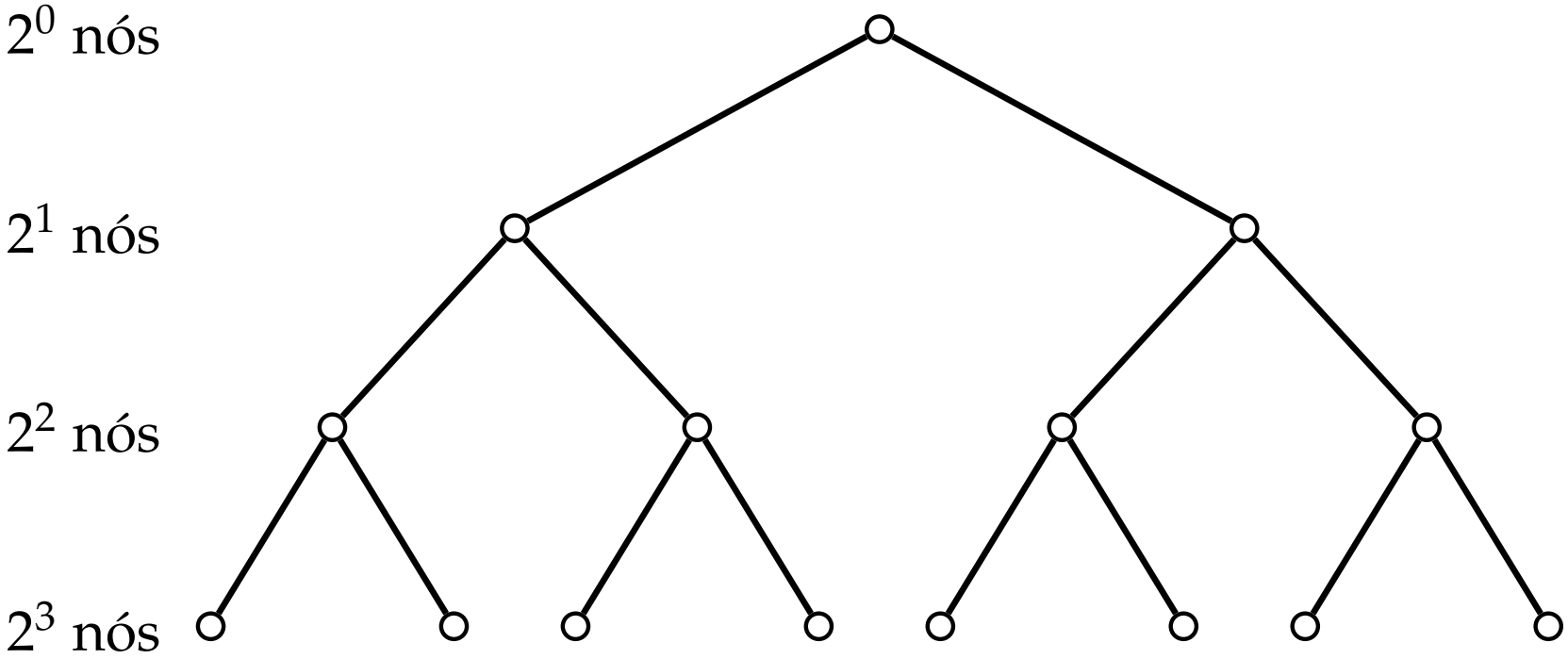
Árvores Binárias

- Árvore binária **cheia**: Se v é um nó tal que alguma subárvore de v é vazia, então v se localiza no último (maior) nível da árvore. Neste caso, v é um nó folha.



Árvores Binárias

- Propriedade: em uma árvore binária **cheia**, o número de nós do nível i é igual a 2^i.



Árvores Binárias

- Como representar árvores binárias?
- Representação sequencial como matriz.

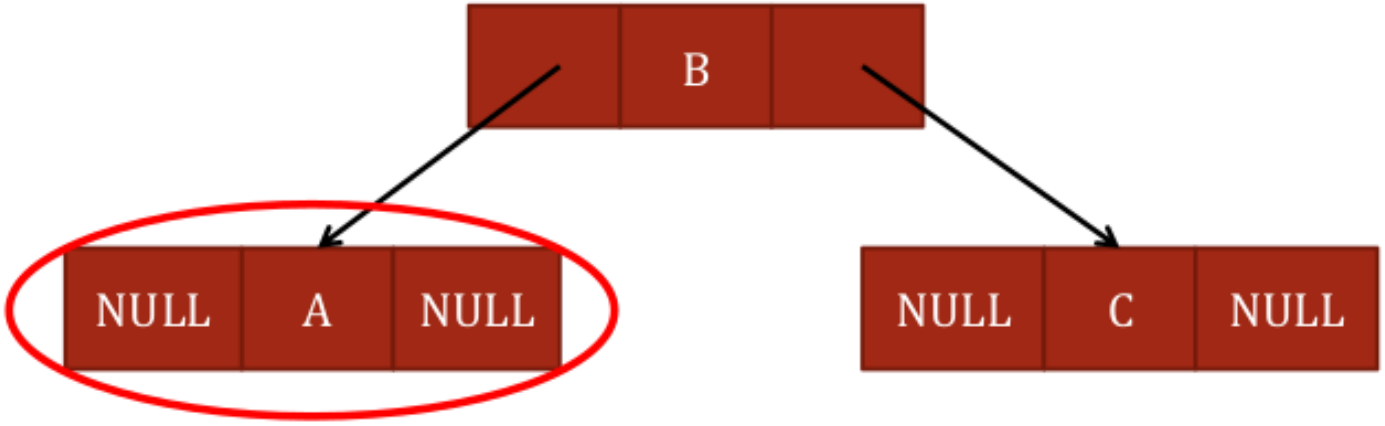
índice	info	esq	dir
0	13	4	2
1	31	6	-1
2	25	7	1
3	12	-1	-1
4	10	5	3
5	2	-1	-1
6	29	-1	-1
7	20	-1	-1

- Nó: estrutura com um campo de informação e dois indicadores para os filhos à esquerda e à direita.
- A raiz é alocada na primeira posição (de índice 0).
- Indicador igual a -1 corresponde a inexistência de filho naquela direção.

- Qual o problema desse tipo de representação?

Árvores Binárias

- Representação encadeada.
- Neste caso, um nó será constituído por três campos:
  - Informação;
  - Ponteiro para nó filho à esquerda;
  - Ponteiro para nó filho à direita.
- Esquematicamente:



Nó da árvore

Árvores Binárias

- TAD `No` para uma árvore binária de caracteres.

```
class No
{
 public:
  No();
  ~No();
  void setEsq(No * p);
  void setDir(No * p);
  void setInfo(char c);
  No * getEsq();
  No * getDir();
  char getInfo();

 private:
  char info;
  No * esq;
  No * dir;
};
```

Árvores Binárias

- Implementação do TAD `No` para uma árvore binária de caracteres.

```
No::No() {}
No::~No() {}

void No::setEsq(No *p)    { esq = p;  }
void No::setDir(No *p)    { dir = p;  }
void No::setInfo(char c) { info = c; }
No * No::getEsq()         { return esq;  }
No * No::getDir()         { return dir;  }
char No::getInfo()        { return info; }
```

Árvores Binárias

- Como representar a árvore binária?
- Basta ter um ponteiro para a raiz da árvore.

```
class ArvBin
{
  public:
    ArvBin();
    ~ArvBin();

    char consultaRaiz();
    void cria(char c, ArvBin *sae, ArvBin *sad);
    bool vazia();
    void imprime();
    bool busca(char c);

  private:
    No * raiz;
};
```

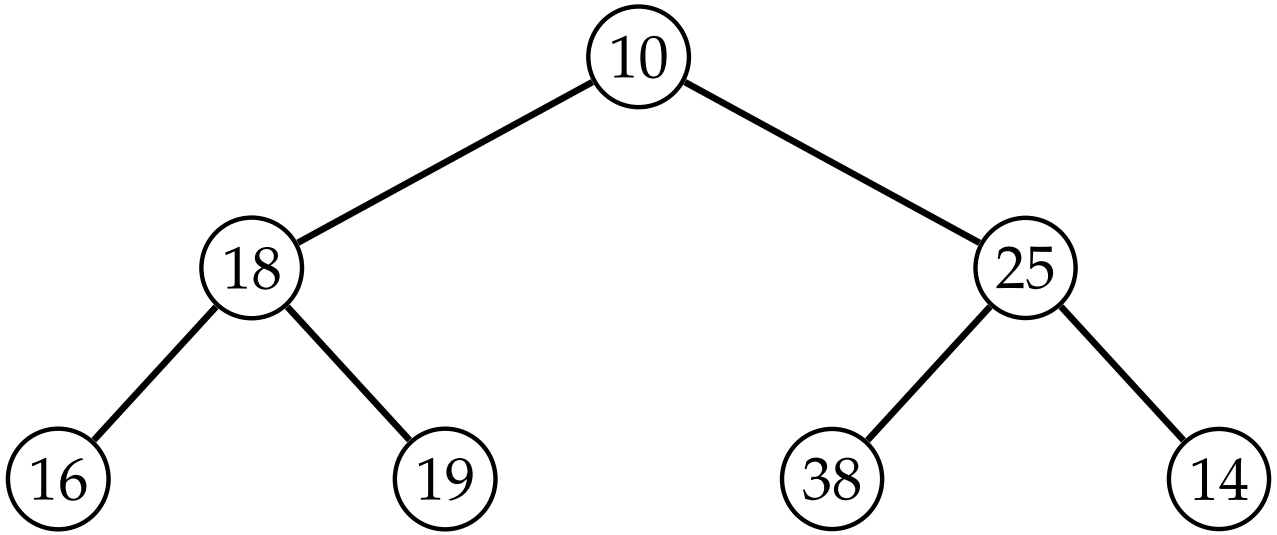
Árvores Binárias

```
ArvBin::ArvBin()
{
  raiz = NULL;
}

char ArvBin::consultaRaiz()
{
  if(raiz != NULL)
    return raiz->getInfo();
  else {
    cout << "Erro: arvore vazia" << endl;
    exit(1);
  }
}

bool ArvBin::vazia()
{
  return (raiz == NULL);
}
```

Árvores Binárias

```
void ArvBin::cria(char c, ArvBin *sae, ArvBin *sad)
{
  No * p = new No();
  p->setInfo(c);
  p->setEsq(sae->raiz);
  p->setDir(sad->raiz);
  raiz = p;
}
```

- Como usar essa função para gerar a seguinte árvore?



Árvores Binárias

- Antes de ver como implementar as operações `imprime()`, `busca()` e o destrutor da classe `ArvBin`, serão apresentados os diferentes algoritmos de percurso em árvore.
- Percurso corresponde a uma visita sistemática a cada um dos nós da árvore.
- Esta é uma das operações básicas relativas à manipulação de árvores.
- Uma árvore é essencialmente uma estrutura não-sequencial.

Percurso em Árvores Binárias

- Para percorrer a árvore deve-se, então, visitar cada um de seus nós.
- Visitar um nó significa operar com a informação do nó.
- Por exemplo: imprimir, atualizar informações etc.
- Percorrer uma árvore significa visitar os seus nós exatamente uma vez.
- Contudo, durante um percurso pode-se passar várias vezes por alguns nós sem visitá-los.

Percurso em Árvores Binárias

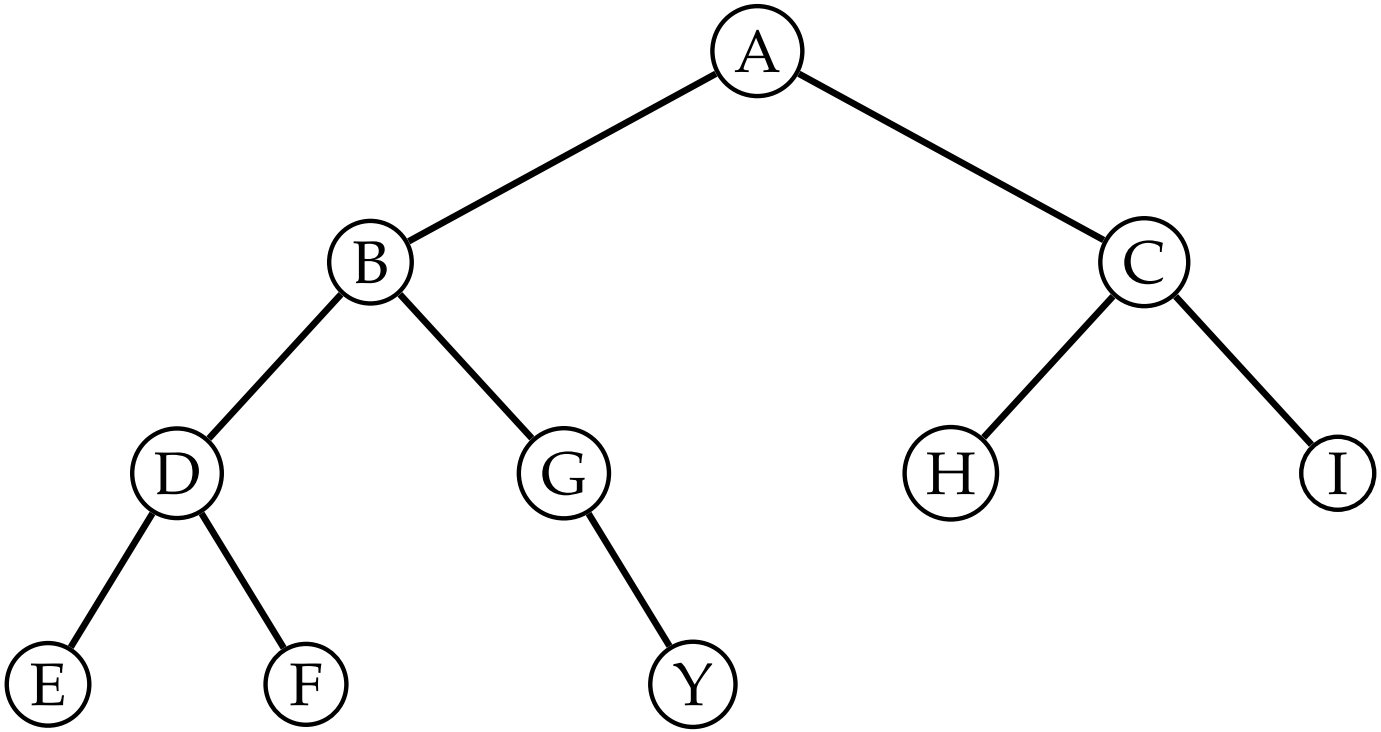
- Passos básicos do percurso.
- Visitar a raiz de cada subárvore de T.
- Visitar as subárvores de T.
- Percorrer as subárvores da esquerda e da direita da raiz.
- Logo, as três operações que compõem o algoritmo são:
  - Visitar a raiz, percorrer a subárvore da esquerda e percorrer a subárvore da direita.
- **Questão**: definir a ordem em que estas operações são realizadas de acordo com o problema.

Percurso em Árvores Binárias

- Algoritmo percurso em profundidade.
  - Seguir tanto quanto possível à esquerda (ou direita);
  - Então mover para trás até a primeira encruzilhada;
  - Seguir um passo para direita (ou esquerda);
  - Novamente, seguir tanto quanto possível para a esquerda (ou direita);
  - Repetir o processo até que todos os nós tenham sido visitados.
- Envolve 3 tarefas:
  - V – Visitar um nó;
  - L – Percorrer subárvore esquerda (left);
  - R – Percorrer subárvore direita (right).
- 3! possibilidades: VLR, LVR, LRV, VRL, RVL, RLV.
- Percurso em **pré-ordem** (ou pré-fixado): VLR
- Percurso em **ordem** simétrica (central ou em-ordem): LVR
- Percurso em **pós-ordem** (ou pós-fixado): LRV

Percurso pré-ordem

- Percurso em pré-ordem (pré-fixado)
  - Visitar a raiz;
  - Percorrer sua subárvore esquerda, em pré-ordem;
  - Percorrer sua subárvore direita, em pré-ordem.



- Ex: A, B, D, E, F, G, Y, C, H, I.

Percursos em ordem e pós-ordem

- Em ordem (simétrica):
  - Percorrer sua subárvore esquerda, em in-ordem;
  - Visitar a raiz;
  - Percorrer sua subárvore direita, em in-ordem.
- Pós-ordem:
  - Percorrer sua subárvore esquerda, em in-ordem;
  - Percorrer sua subárvore direita, em in-ordem;
  - Visitar a raiz.
- Observe a natureza **recursiva** dos algoritmos de percurso.
- Exercício: execute os percursos em ordem e pós-ordem na árvore binária do exemplo anterior. Qual a sequência de visita aos nós?

Árvores Binárias

- Agora, a função `imprime` do TAD `ArvBin` pode ser implementada utilizando, por exemplo, um **percurso pré-ordem**.

```
void ArvBin::imprime()
{
  auxImprime(raiz);
}

void ArvBin::auxImprime(No * p)
{
  if(p != NULL)
  {
          cout << p->getInfo() << endl;
          auxImprime( p->getEsq() );
          auxImprime( p->getDir() );
  }
}
```

Árvores Binárias

- Mudanças na classe `ArvBin`:

```
class ArvBin
{
  private:
    No *raiz;
    void auxImprime(No * p);

    // etc ...

  public:
    // etc ...
};
```

- A função `auxImprime` é que realiza toda a tarefa de imprimir o conteúdo da árvore binária.

Árvores Binárias

```
void ArvBin::auxImprime(No * p) // em ordem
{
  if(p != NULL)
  {
    auxImprime( p->getEsq() );
    cout << p->getInfo() << endl;
    auxImprime( p->getDir() );
  }
}


void ArvBin::auxImprime(No * p) // pos-ordem
{
  if(p != NULL)
  {
    auxImprime( p->getEsq() );
    auxImprime( p->getDir() );
    cout << p->getInfo() << endl;
  }
}
```

Árvores Binárias

```
bool ArvBin::busca(char ch)
{
  return auxBusca(raiz, ch);
}
```

- Mudanças na classe `ArvBin`:

```
class ArvBin
{
  private:
    No *raiz;

    bool auxBusca(No * p, char ch);

    // etc ...
};
```

- A função `auxBusca` é que realiza toda a tarefa de busca.
- Como fica a implementação dessa função `auxBusca`?

Árvores Binárias

- A função busca procura a chave na árvore seguindo um percurso **pré-ordem**.

```
bool ArvBin::auxBusca(No *p, char ch)
{
  if (p == NULL)
    return false;
  else if ( p->getInfo() == ch )
    return true;
  else if ( auxBusca(p->getEsq(),ch) )
    return true;
  else
    return auxBusca(p->getDir(),ch);
}
```

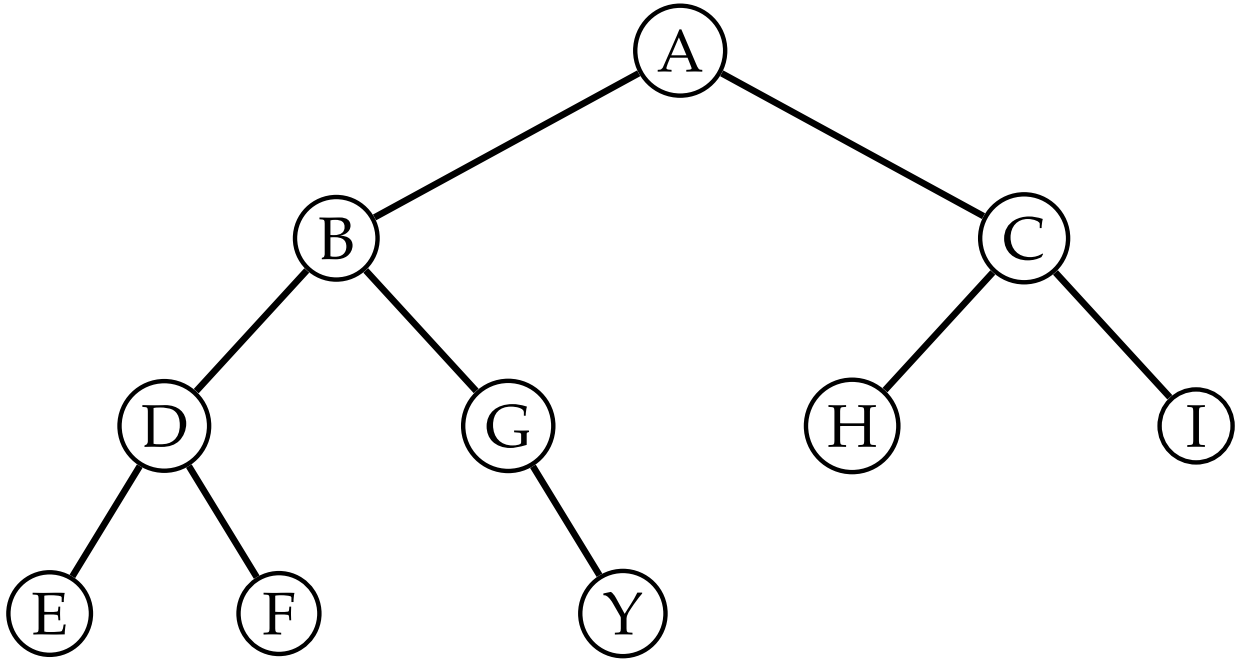
Árvores Binárias

- Destrutor.

```
ArvBin::~ArvBin()
{
  raiz = libera(raiz);
}

No* ArvBin::libera(No *p)
{
  if (p != NULL)
  {
    p->setEsq( libera(p->getEsq()) );
    p->setDir( libera(p->getDir()) );
    delete p;
    p = NULL;
  }
  return NULL;
}
```

Árvores Binárias

- Percurso em **largura**.
- Corresponde a visitar cada nó começando no nível mais alto (ou mais baixo) e movendo para baixo (ou para cima) nível a nível, visitando nós em cada nível da esquerda para a direita (ou da direita para a esquerda).



- Ex: A, B, C, D, G, H, I, E, F, Y.

Árvores Binárias

- A implementação de um percurso em largura usa uma fila.
- Por exemplo, para um percurso em extensão de cima para baixo, da esquerda para a direita usando fila.
  - Entra com o nó raiz na fila;
  - O nó do início da fila é visitado (sai da fila);
  - Quando um nó é visitado, seus filhos (se houverem) são colocados no final da fila.
- Para um nó j no nível n, seus filhos estão no nível $n + 1$.
- Após a visita do nó j, seus filhos são colocados no final da fila.
- Portanto, eles só serão visitados depois que todos os nós do nível n forem visitados.
- Assim a restrição de que todos os nós no nível n precisam ser visitados antes de visitar quaisquer nós no nível $n + 1$ será satisfeita.

Árvores Binárias

- Algoritmo do percurso em largura:

```
Se árvore não está vazia
{
  Coloca nó raiz na fila;
  Enquanto a fila não está vazia
  {
    Visita o nó do início da fila;
    Retira o nó da fila;
    Se tem filho à esquerda
      Coloca o nó na fila;
    Se tem filho à direita
      Coloca o nó na fila;
  }
}
```

Árvores Binárias

- **Exercício**: desenvolver uma operação do TAD `arvBin` que calcula a altura da árvore binária. Utilize uma função auxiliar privada que seja recursiva.

```
int ArvBin::altura();
```

- Classe:

```
class ArvBin
{
  public:
    // ...
    int altura();

  private:
    // ...
    int auxAltura(No * p);

};
```

Árvores Binárias

```
int ArvBin::auxAltura(No *p)
{
  return auxAltura(raiz);
}

int ArvBin::auxAltura(No *p)
{
  int he, hd;
  if (p == NULL)
    return -1;
  else
  {
    he = auxAltura( p->getEsq() );
    hd = auxAltura( p->getDir() );
    if(he > hd)
      return 1 + he;
    else
      return 1 + hd;
  }
}
```

Árvores Binárias

```
int ArvBin::auxAltura(No *p)
{
  int he, hd;
  if (p == NULL)
    return -1;
  else
  {
    he = auxAltura( p->getEsq() );
    hd = auxAltura( p->getDir() );
    return 1 + Maior(he,hd);
  }
}
```

Árvores Binárias

- **Exercício**: desenvolver um algoritmo para realizar o percurso em largura em uma árvore binária de caracteres e imprimir as informações de todos os seus nós.
- Dica: será preciso utilizar uma fila (encadeada) cujos elementos são ponteiros para os nós da árvore binária. Veja a definição das classes `FilaNoArv` e `NoF` a seguir.

Árvores Binárias

```
class NoF
{
  public:
    NoF(){};
    ~NoF(){};
    void setInfo(No *q);
    void setProx(NoF *p);
    No *getInfo();
    NoF *getProx();
  private:
    No *info;  // info eh um ponteiro para um No da
         arvore
    NoF *prox; // ponteiro para o proximo No da
        fila
};
```

Árvores Binárias

```
class FilaNoArv
{
  public:
    FilaNoArv();
    ~FilaNoArv();
    No* consultaInicio();
    void entra(No *q);
    void sai();
    bool vazia();

  private:
    NoF *c; // comeco da fila
    NoF *f; // fim da fila
};
```

Árvores Binárias

```
void ArvBin::percLargura()
{
  No * p = raiz;
  No *q;
  FilaNoArv fila;
  if (p != NULL)
  {
    fila.entra(p);
    while (!fila.vazia())
    {
      q = fila.consultaInicio();
      cout << q->getInfo() << endl;
      fila.sai();
      if(q->getEsq() != NULL)
        fila.entra( q->getEsq() );
      if(q->consultaDir() != NULL)
        fila.entra( q->getDir() );
    }
  }
}
```

Exercícios

1. Implemente uma operação para contar o número de nós de uma árvore binária.

$$nNos(arv) = \begin{cases} 0, & \text{se } arv \text{ vazia} \\ 1 + nNos(sae) + nNos(sad), & \text{caso contrário} \end{cases}$$

2. Implemente uma operaçõa para contar e retornar o número de folhas de uma árvore binária.

$$nFolhas(arv) = \begin{cases} 0, & \text{se } arv \text{ vazia} \\ 1, & \text{se } ehFolha(arv) \\ nFolhas(sae) + nFolhas(sad), & \text{caso contrário} \end{cases}$$

A função `ehFolha(p)` retorna true se `p` é um nó folha ou `false` caso contrário.

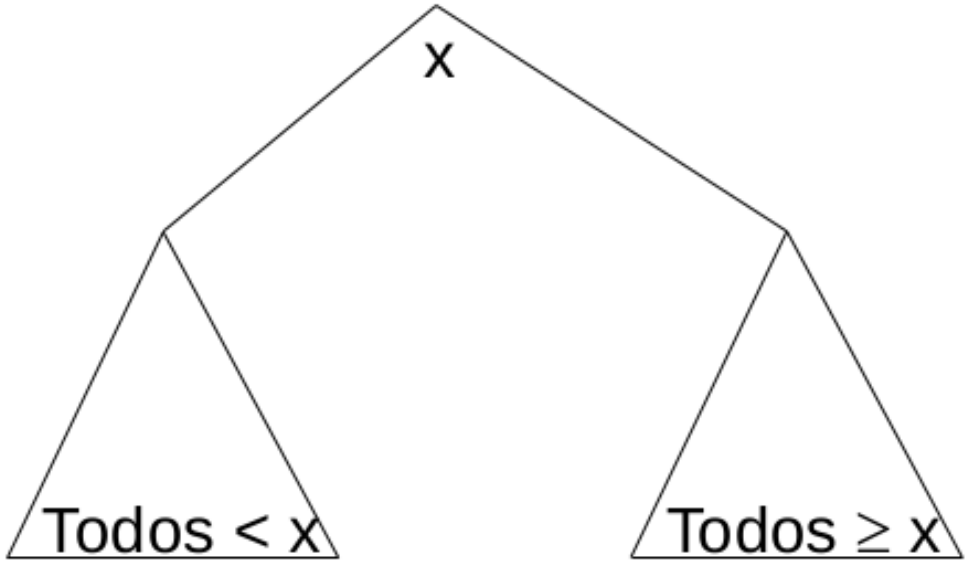
Exercícios

3. Implemente uma operação para verificar se uma árvore binária é cheia. O número de nós n de uma árvore binária cheia é dado por $n = 2^{h+1}-1$, onde h é altura da árvore binária. Também, pode-se considerar que uma árvore binária é cheia quando o número de nós do último nível (número de folhas) da árvore binária é 2^h.
4. Implemente uma operação para excluir todas as folhas de uma árvore binária, deixando a raiz e os nós intermediários nos respectivos lugares. Dica: usar o percurso em pré-ordem.
5. No exercício 3 a árvore binária é percorrida 2 vezes (uma para calcular o número de folhas e outra para a sua altura). Desenvolver a mesma operação porém percorrendo a árvore apenas uma única vez.
6. Reescrever a operação de percurso em pré-ordem usando uma pilha ao invés de recursão.

Árvore Binária de Busca

Árvore Binária de Busca

- Definição: uma árvore binária de busca (ABB) é uma árvore binária na qual cada nó possui uma chave comparável e que satisfaz a seguinte restrição: a chave em qualquer nó é
  - maior do que as chaves de todos os nós da sub-árvore à esquerda e
  - menor (ou igual) às chaves de todos os nós da sub-árvore à direita.



Árvore Binária de Busca

- A representação de uma ABB é idêntica à representação de árvores binárias (TAD `ArvBin`).
- Portanto, a implementação deste TAD usa o mesmo TAD `No` apresentado anteriormente.
- Entretanto, algumas operações precisam ser repensadas:
  - busca
  - inserção (*)
  - remoção
- Essas operações **devem** explorar a **propriedade de ordenação das ABBs**.

TAD Árvore Binária de Busca

```
class ArvBinBusca
{
private:
  No *raiz;

  bool auxBusca(No * p);

public:
  ArvBinBusca();
  ~ArvBinBusca();

  char consultaRaiz();
  bool vazia();
  bool busca(int x);
  void remove(int x);

  // outras operacoes

};
```

Busca na Árvore Binária

Revisão

- Implementação da busca para **Árvore Binária** (!)

```
bool ArvBin::auxBusca(No *p, char C)
{
  if (p == NULL)
    return false;
  else if (p->consultaInfo() == C)
    return true;
  else if (AuxBusca(p->consultaEsq(),C))
    return true;
  else
    return AuxBusca(p->consultaDir(),C);
}
```

Busca na Árvore Binária de Busca

- Como implementar essa operação para a **Árvore Binária de Busca**?
- Na ABB deve-se considerar a **propriedade** de ordenação para realizar a busca de forma mais **eficiente**.
- Ideia: compara-se o valor procurado com a informação do nó raiz:
  - Se igual, achou
  - Se menor, buscar na sub-árvore da esquerda
  - Se maior, buscar na sub-árvore da direita

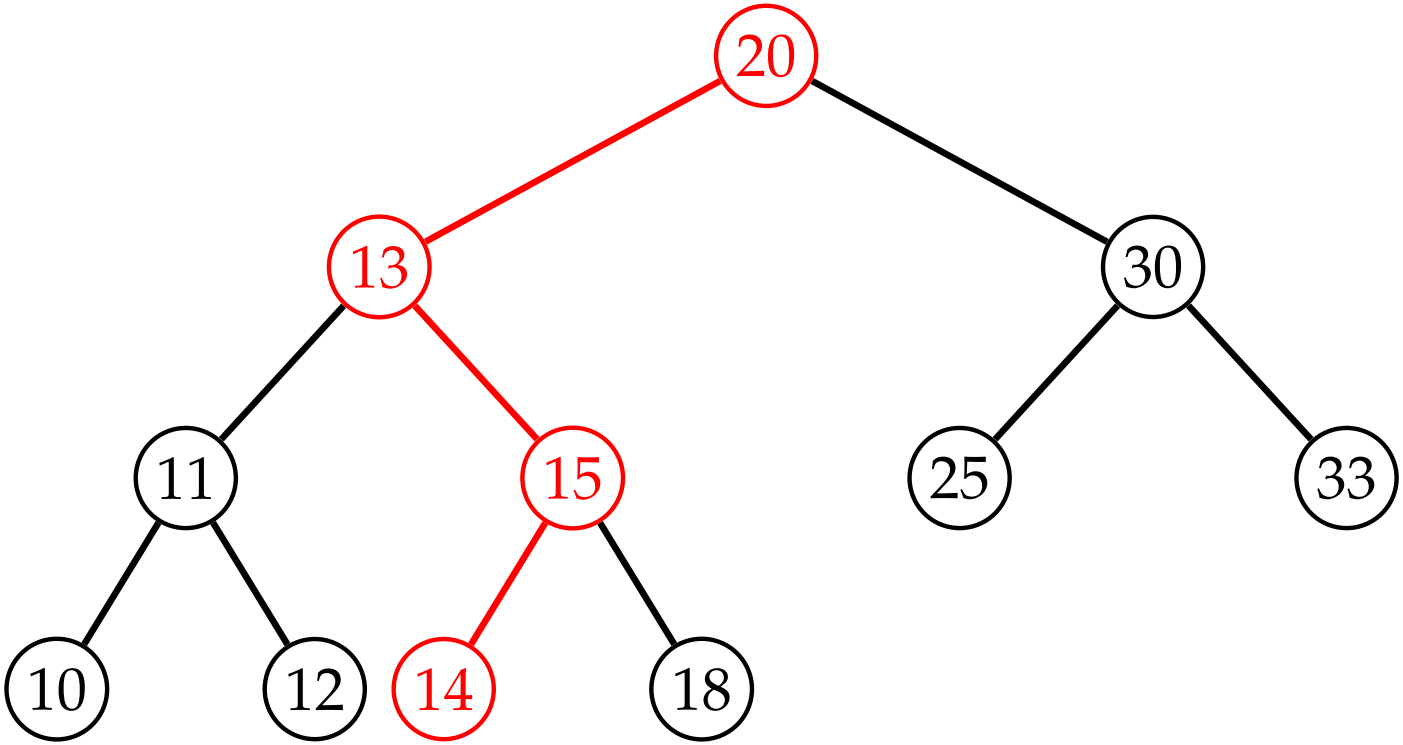
TAD Árvore Binária de Busca

- Implementação da operação de busca

```
bool ArvBinBusca::busca(int x)
{
  return auxBusca(raiz, x);
}

bool ArvBinBusca::auxBusca(No *p, int x)
{
  if (p == NULL)
    return false;
  else if (p->consultaInfo() == x)
    return true;
  else if (x < p->consultaInfo())
    return auxBusca(p->consultaEsq, x);
  else // x > p->consultaInfo()
    return auxBusca(p->consultaDir(), x);
}
```

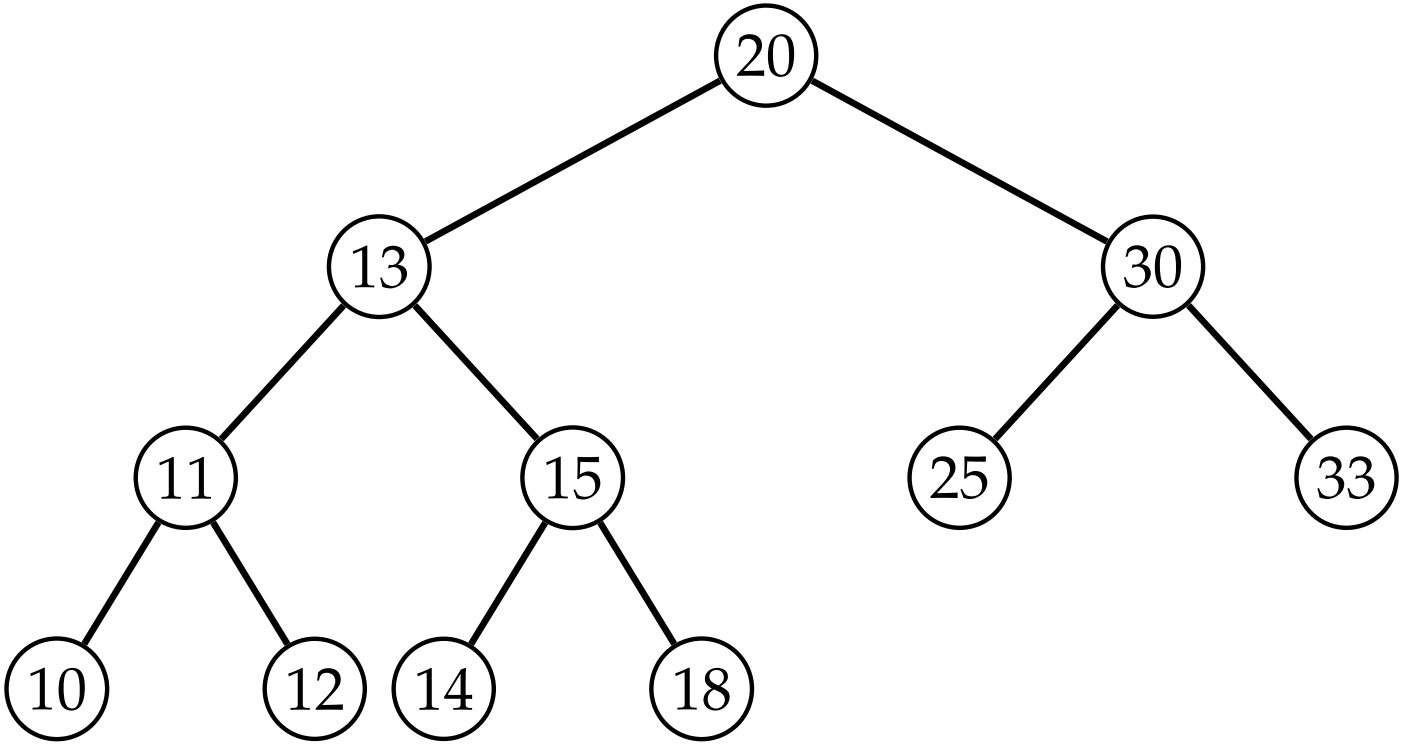
Busca na Árvore Binária de Busca

- Exemplo do caminho percorrido para encontrar a chave 14.



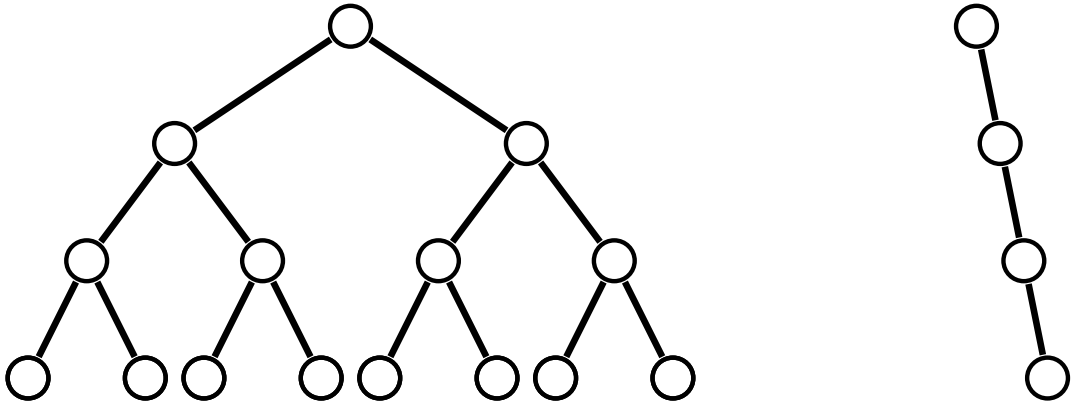
Busca na Árvore Binária de Busca

- Apresentar os caminhos percorridos para buscar as seguintes chaves: 18, 11, 10, 25, 35, 7



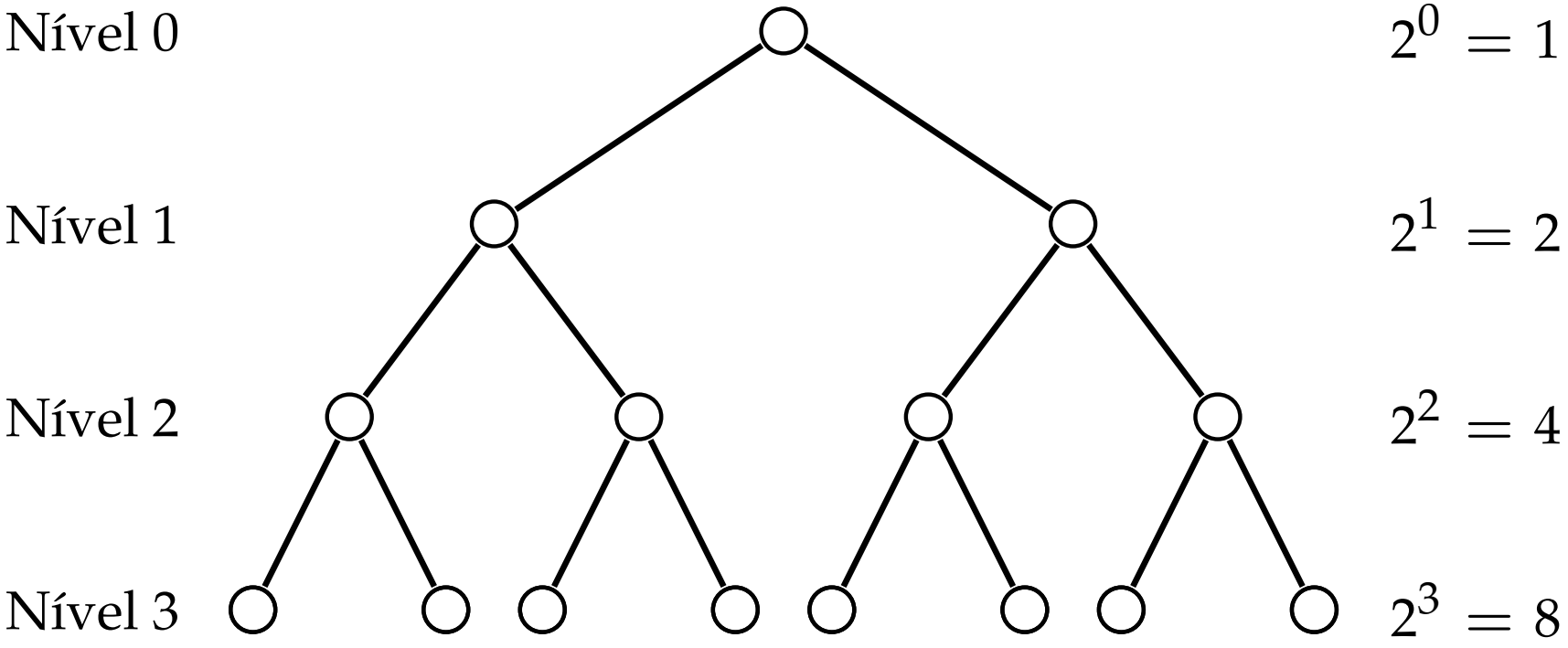
Busca na Árvore Binária de Busca

- O tempo de execução dos algoritmos em ABB depende da forma das árvores, que por sua vez, depende da ordem na qual as chaves são inseridas.
- No melhor caso, tem-se uma árvore de N nós perfeitamente balanceada com $\log_2 N$ nós entre a raiz e as folhas.
- No pior caso, tem-se uma árvore com N nós entre a raiz e as folhas.
- Complexidade: o número de comparações realizados na busca é proporcional à altura h da árvore, isto é, $O(h)$.



Árvore Binária de Busca

- Qual é a altura h de uma árvore binária com N nós?
- Altura h = maior nível.
- Nível $k \rightarrow 2^k$ nós.
- Melhor situação é quando a árvore binária é cheia (balanceada):



Árvore Binária de Busca

- Para uma árvore binária cheia:
  - Nível $k \rightarrow 2^k$ nós.
  - Propriedade: o número de nós de um nível k qualquer é igual a 1 mais a soma de todos os nós dos níveis anteriores.

nível	número de nós
0	$2^0 = 1$
1	$2^1 = 2(1+1)$
2	$2^2 = 4(1+3)$
3	$2^3 = 8(1+7)$
...	...
k	$2^k = 1 + \sum_{i=0}^{k-1} 2^i$
...	...
h+1	$2^{h+1} = 1 + \sum_{i=0}^{h} 2^i$

Árvore Binária de Busca

- Assim, tem-se:

$$2^{h+1} = 1 + \sum_{i=0}^{h} 2^i$$

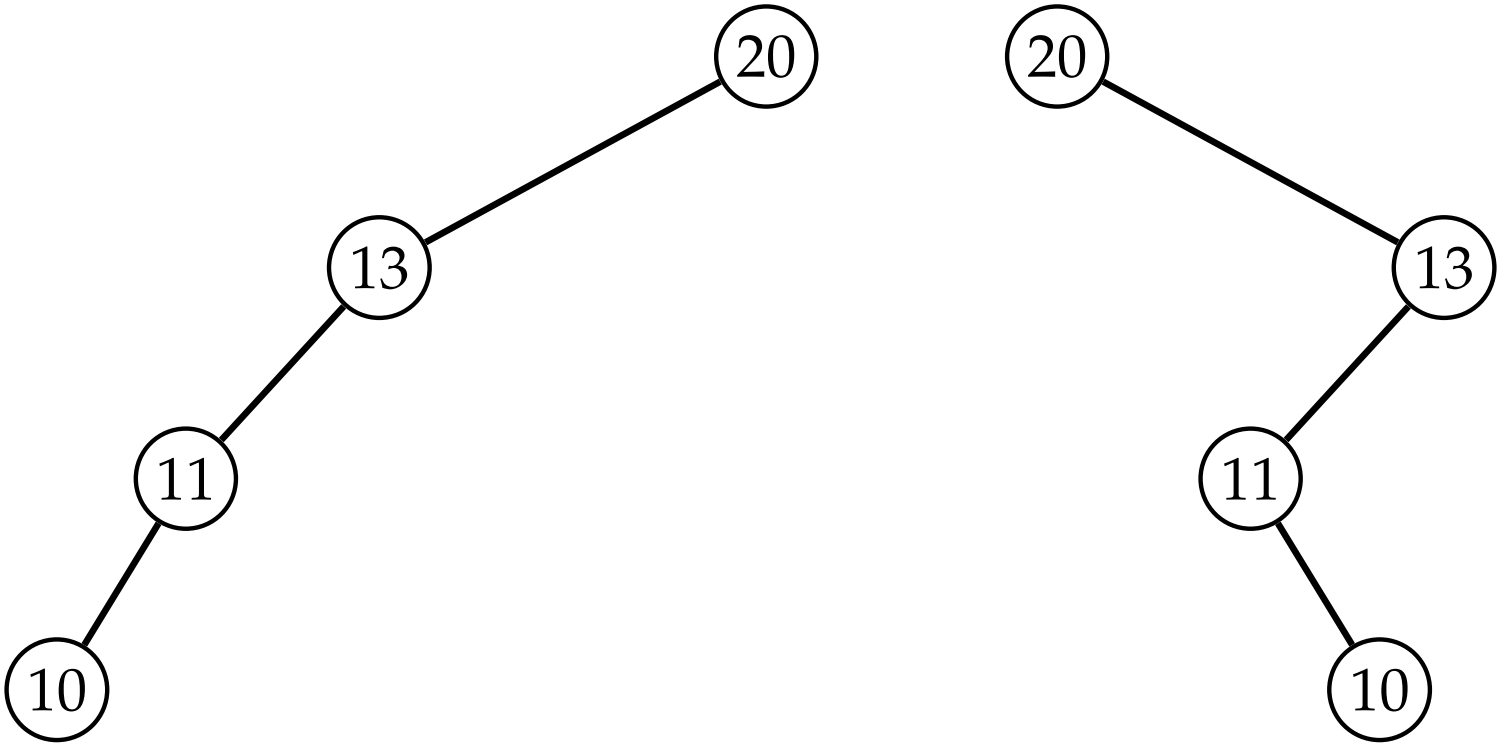
- Note que $\sum_{i=0}^{h} 2^i$ é o número total de nós da árvore, que é N.
- Portanto

$$\begin{aligned} 2^{h+1} &= 1 + N \\ \log_2 (2^{h+1}) &= \log_2 (1 + N) \\ h + 1 &= \log_2 (1 + N) \\ h &= \log_2 (1 + N) - 1 \end{aligned}$$

- Eliminando as constantes, conclui-se que, em uma árvore binária de busca balanceada, a altura h é proporcional a $\log_2 (N)$.

Árvore Binária de Busca

- Pior caso: árvore binária **desbalanceada** (degenerada).
- Neste caso a altura h é proporcional ao número de nós N.



Árvore Binária de Busca

- Em resumo, sobre a complexidade na ABB:
- O número de comparações realizadas nas operações é proporcional à altura h da árvore: $O(h)$.
- A altura ABB é no mínimo $\log_2 N$ e no máximo N.
- Portanto, a complexidade das operações em uma árvore binária de busca é $O(\log_2 N)$ na **melhor situação** (balanceada) e $O(N)$ na **pior situação** (degenerada), onde N é o número de nós da árvore.

Árvore Binária de Busca

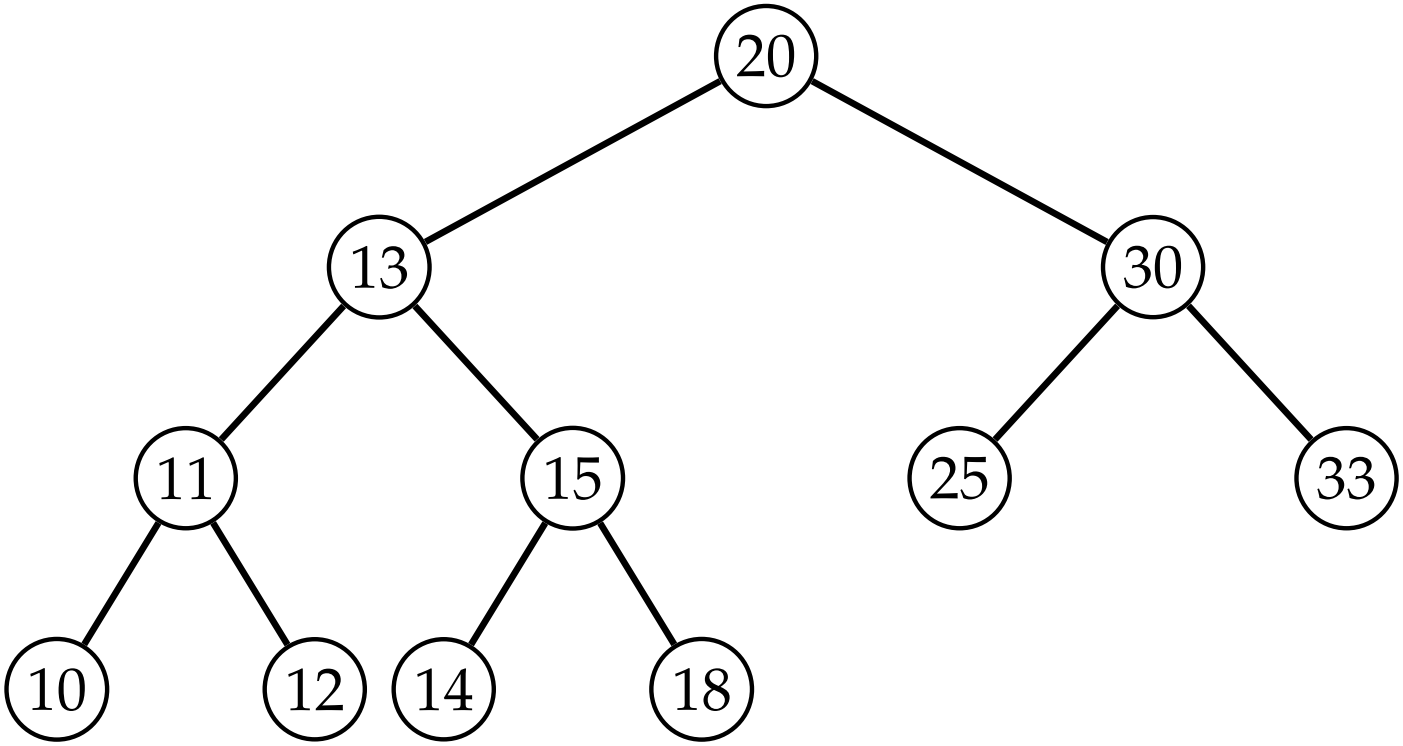
Inserção

- Para inserir um novo nó com valor y, deve-se percorrer a árvore buscando a chave y, até o nó que será o seu pai, isto é, o nó que não apresentar filho na sequência natural do percurso (ou filho = `NULL`).
- Em seguida, basta incluir um nó folha contendo y.

Árvore Binária de Busca

Inserção

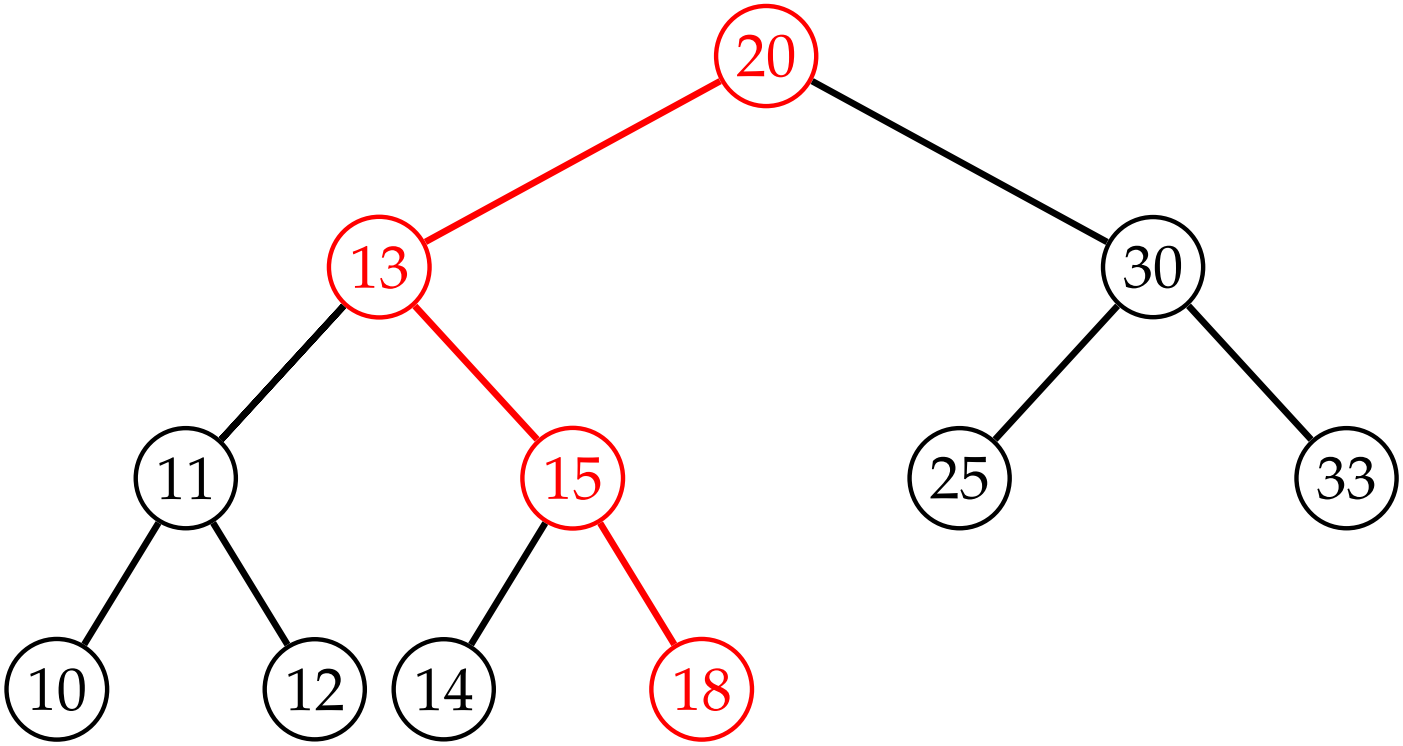
- Exemplo: inserir os valores 17 e 27 na árvore abaixo.



Árvore Binária de Busca

Inserção

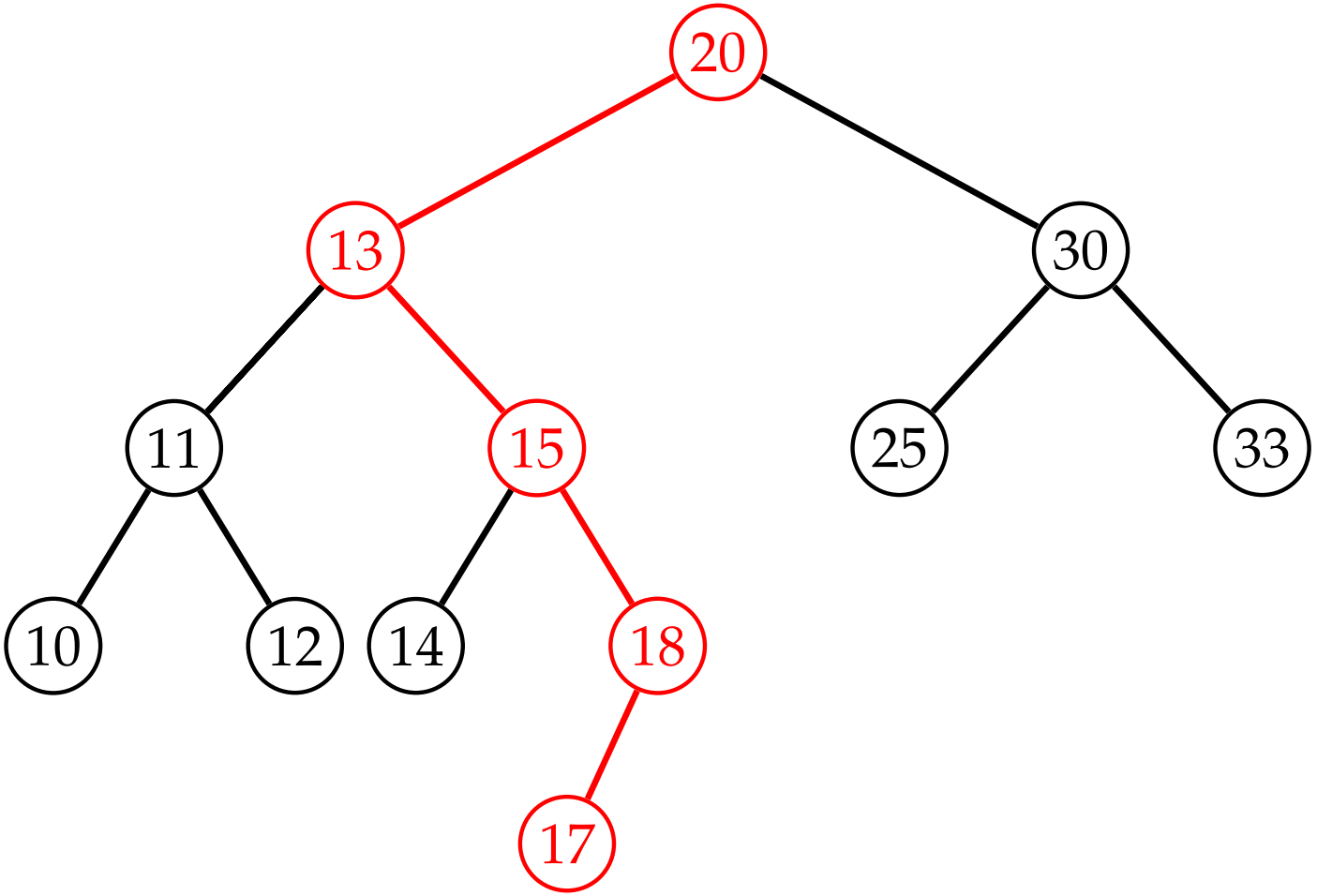
- Exemplo: inserir o valor 17



Árvore Binária de Busca

Inserção

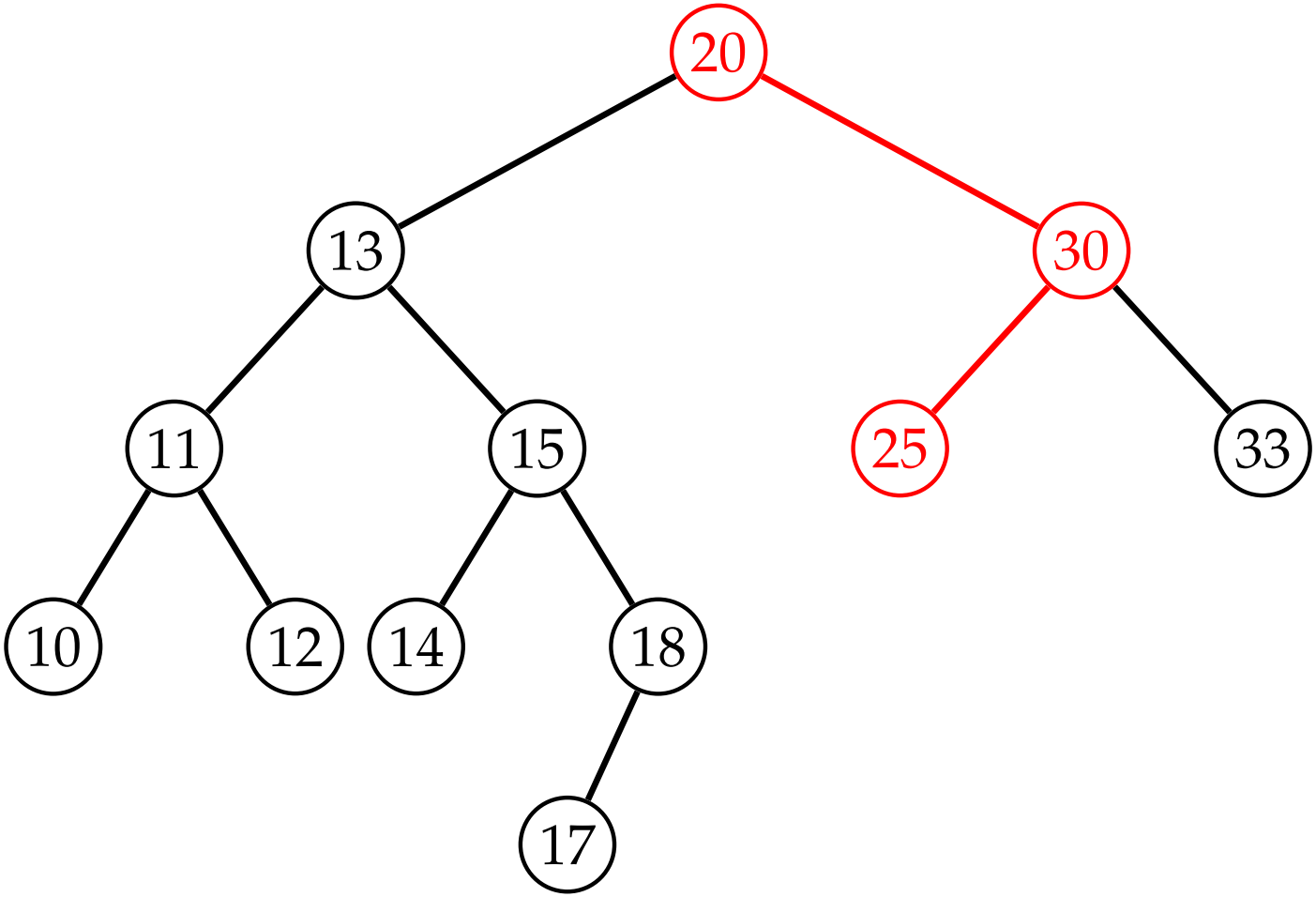
- Exemplo: inserir o valor 17



Árvore Binária de Busca

Inserção

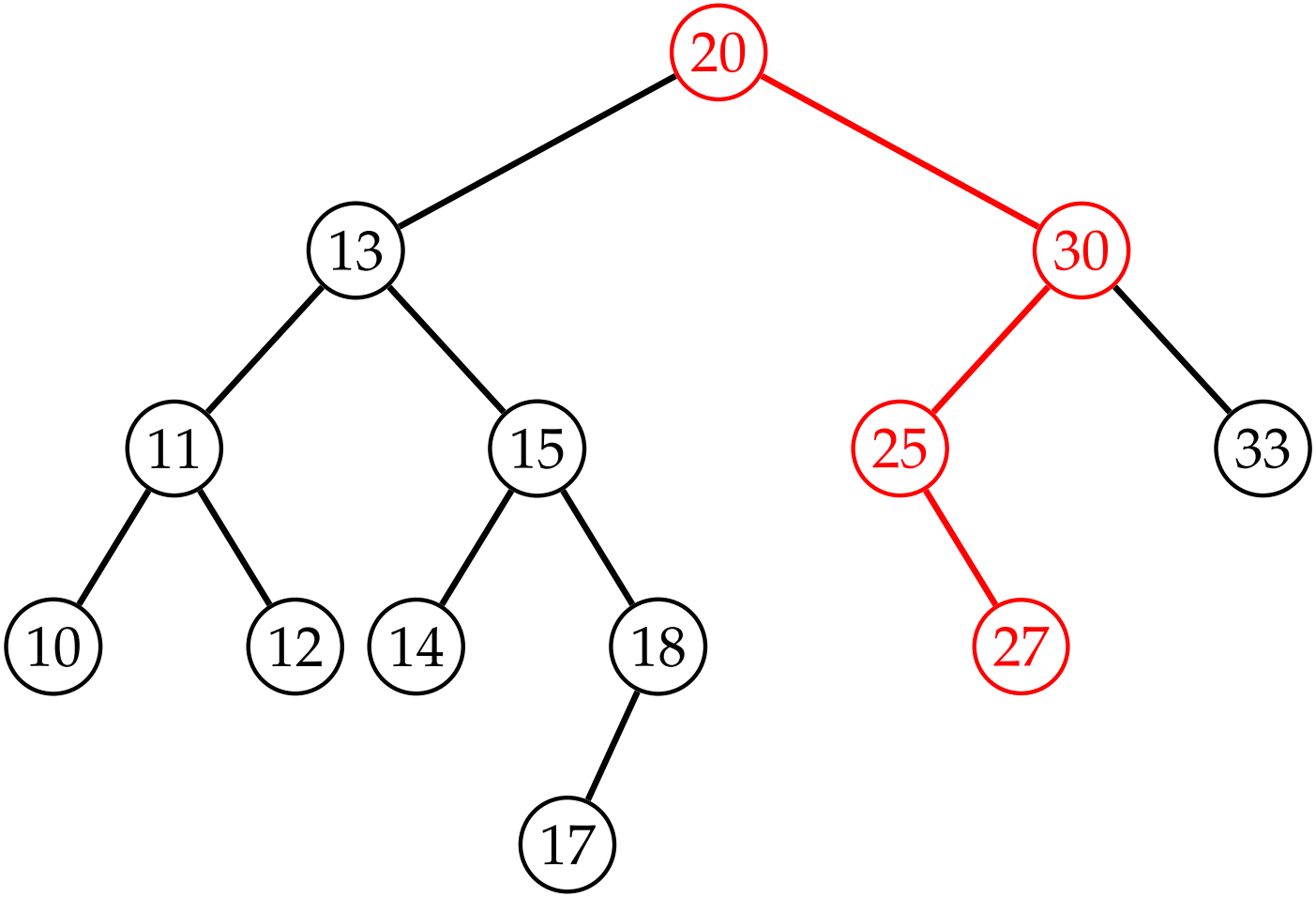
- Em seguida, inserir o valor 27



Árvore Binária de Busca

Inserção

- Em seguida, inserir o valor 27



Árvore Binária de Busca

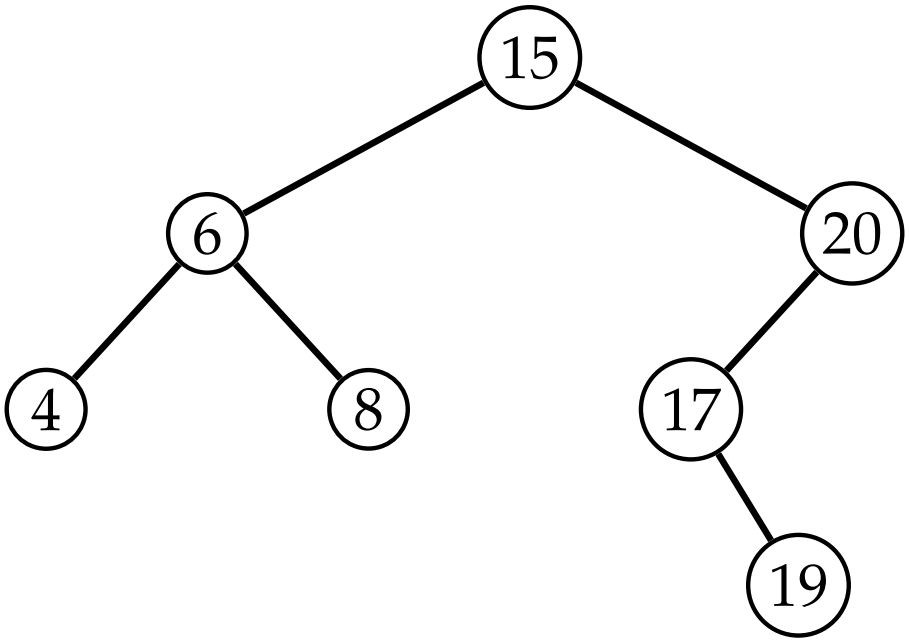
Exemplo

- Considere uma ABB inicialmente vazia.
- Inserir as chaves $15, 6, 20, 17, 8, 4$ e 19
- Qual o resultado após essas inserções?

Árvore Binária de Busca

Exemplo

- Considere uma ABB inicialmente vazia.
- Inserir as chaves $15, 6, 20, 17, 8, 4$ e 19
- Qual o resultado após essas inserções?



TAD Árvore Binária de Busca

Inserção

```
class ArvBinBusca
{
private:
  //...
  No* auxInsere(No *p, int x);
  //...

public:
  //...
  void insere(int x);
  //...
};
```

- A operação pública tem a seguinte implementação

```
void ArvBinBusca::insere(int x)
{
  raiz = auxInsere(raiz, x);
}
```

TAD Árvore Binária de Busca

Inserção

```
No* ArvBinBusca::auxInsere(No *p, int x)
{
  if(p == NULL)
  {
    p = new No();
    p->setInfo(x);
    p->setEsq(NULL);
    p->setDir(NULL);
  }
  else if(x < p->getInfo())
    p->setEsq( auxInsere(p->getEsq(), x) );
  else
    p->setDir( auxInsere(p->getDir(), x) );

  return p;
}
```

Árvore Binária de Busca

Menor e maior

- Pode-se implementar a busca pelo menor (maior) de forma recursiva ou iterativa.
- Descrição **recursiva** para encontrar o **menor**:
  - Se a sub-árvore à esquerda da raiz é vazia, então o menor está na raiz.
  - Se a sub-árvore à esquerda da raiz não é vazia, então o menor está na sub-árvore à esquerda.
- Para encontrar o maior valor, basta trocar esquerda por direita na descrição acima.

Árvore Binária de Busca

Menor e maior

```
int ArvBinBusca::min()
{
  return auxMin(raiz);
}

int ArvBinBusca::auxMin(No *p)
{
  if(p != NULL)
  {
    if(p->getEsq() == NULL)
      return p->getInfo();
    else
      return auxMin( p->getEsq() );
  }
}
```

Árvore Binária de Busca

Remoção

- A operação mais complicada em uma ABB é a de remover um determinado nó.
- Quando deseja-se remover um nó qualquer da árvore, este nó pode ser:
  - uma folha;
  - um nó que possui apenas 1 filho;
  - ou um nó que possui os 2 filhos.
- Antes de estudar o caso mais geral, considere o caso de remover o menor valor da ABB.

Árvore Binária de Busca

```
void ArvBinBusca::removeMin()
{
  raiz = auxRemoveMin(raiz);
}

No * ArvBinBusca::auxRemoveMin(No * p)
{
  if(p != NULL)
  {
    if (p->getEsq() == NULL) {
      No * r = p->getDir();
      delete p;
      return r;
    }

    p->setEsq( auxRemoveMin(p->getEsq()) );
    return p;
  }
}
```

Árvore Binária de Busca

Remoção

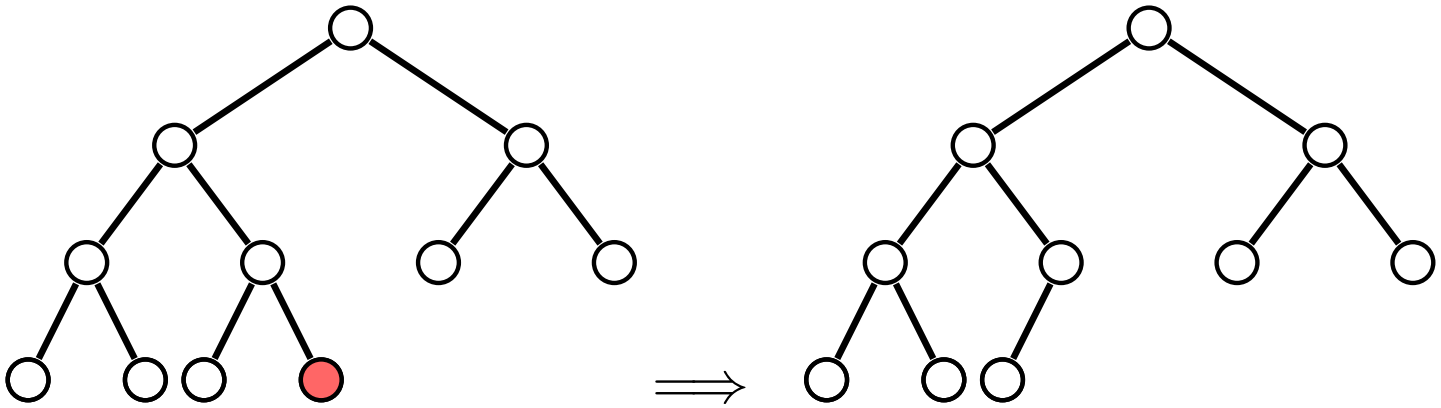
- Para remover um nó qualquer da árvore, serão criadas 3 funções para tratar de cada um dos casos mencionados.
- Ideia geral para remover um nó com a chave `x`:
  - se a árvore é vazia, retorna `NULL`
  - senão se `x < p->consultaInfo()`, remove x na SAE
  - senão se `x > p->consultaInfo()`, remove x na SAD
  - senão se o nó é folha, então `removeFolha()`
  - senão se o nó possui apenas 1 filho, então `removeNo1Filho()`
  - senão (o nó possui 2 filhos), `removeNo2Filhos()`
- Primeiro serão apresentadas as implementações das funções que removem o nó de acordo com o caso em questão, isto é, `removeFolha()`, `removeNo1Filho()` e `removeNo2Filhos()`.

Árvore Binária de Busca

Remove nó folha

- A função `removeFolha(No * p)` recebe um ponteiro para um nó que é uma folha e então remove o nó da árvore.
- O ponteiro do pai é ajustado para `NULL` e o nó é removido.
- É preciso ter certeza de que `p` aponta para um nó folha.

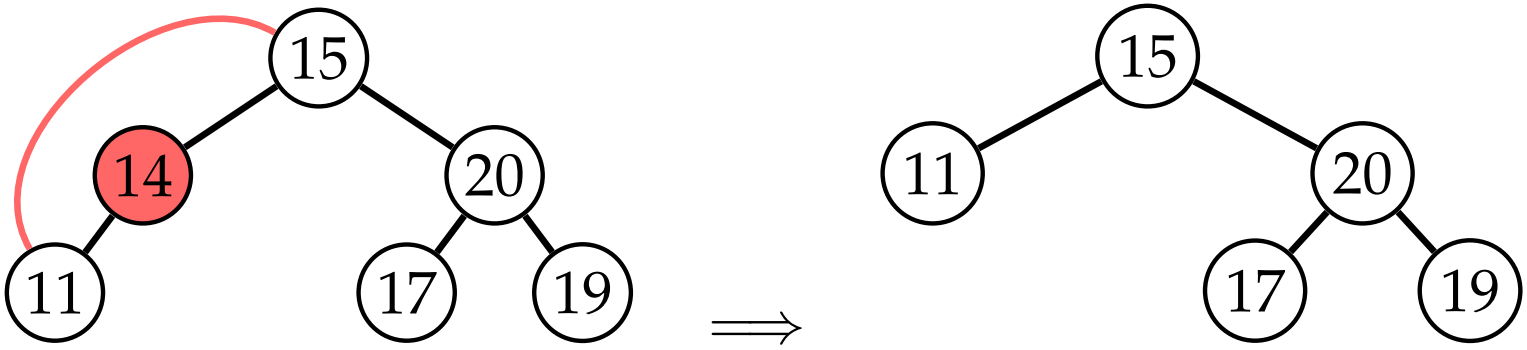
```
No * ArvBinBusca::removeFolha(No * p)
{
  delete p;
  return NULL;
}
```



Árvore Binária de Busca

Remove nó com 1 filho

- Ponteiro do pai do nó a ser removido é reajustado para apontar para o filho do nó a ser removido.
- Ou seja, o pai vai apontar para o "neto"(que passa a ser filho).
- Desse modo, descendentes do nó em questão são elevados em 1 nível.
- Exemplo: remover o nó 14.



Árvore Binária de Busca

Remove nó com 1 filho

- A função `removeNo1Filho()` é usada para um remover um nó `p` que só possua um único filho.
- É preciso **ter certeza** de que `p` só possui um filho para usar essa função.

```
No* ArvBinBusca::removeNo1Filho(No *p)
{
  No *aux;
  if(p->getEsq() == NULL)
    aux = p->getDir();
  else
    aux = p->getEsq();

  delete p;
  return aux;
}
```

Árvore Binária de Busca

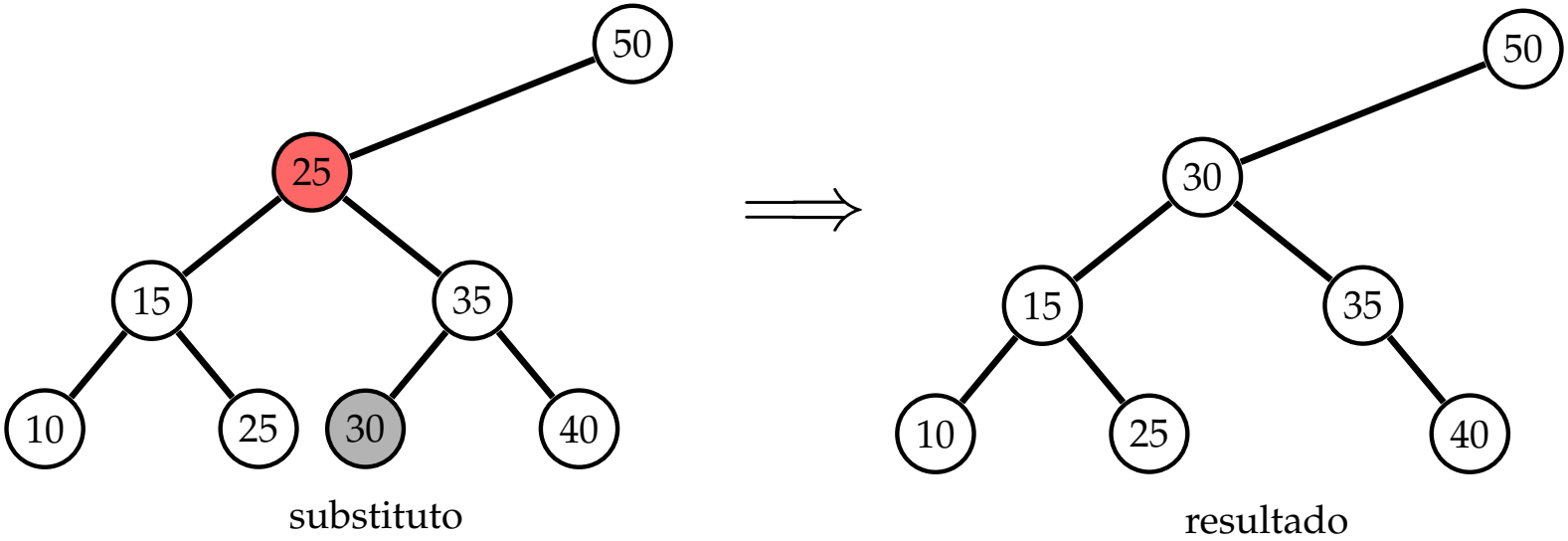
Remove nó com 2 filhos

- Se o nó a ser removido tem 2 filhos:
  - **Remover fazendo cópia.**
  - Remover fazendo junção (*merge*). Não será visto.
- Remoção por cópia (Thomas Hibbard e Donald Knuth): substituir o nó a ser removido pelo menor nó de sua sub-árvore à direita e "ajustar ponteiros".
- Etapas da remoção por cópia:
  1. Buscar substituto (menor nó da sua sub-árvore à direita).
  2. Trocar a informação do nó a ser removido com a do substituto.
  3. Remover o substituto (essa remoção faz o ajuste dos ponteiros).

Árvore Binária de Busca

Remove nó com 2 filhos

- Exemplo: excluir o nó 25.



Árvore Binária de Busca

Remove nó com 2 filhos

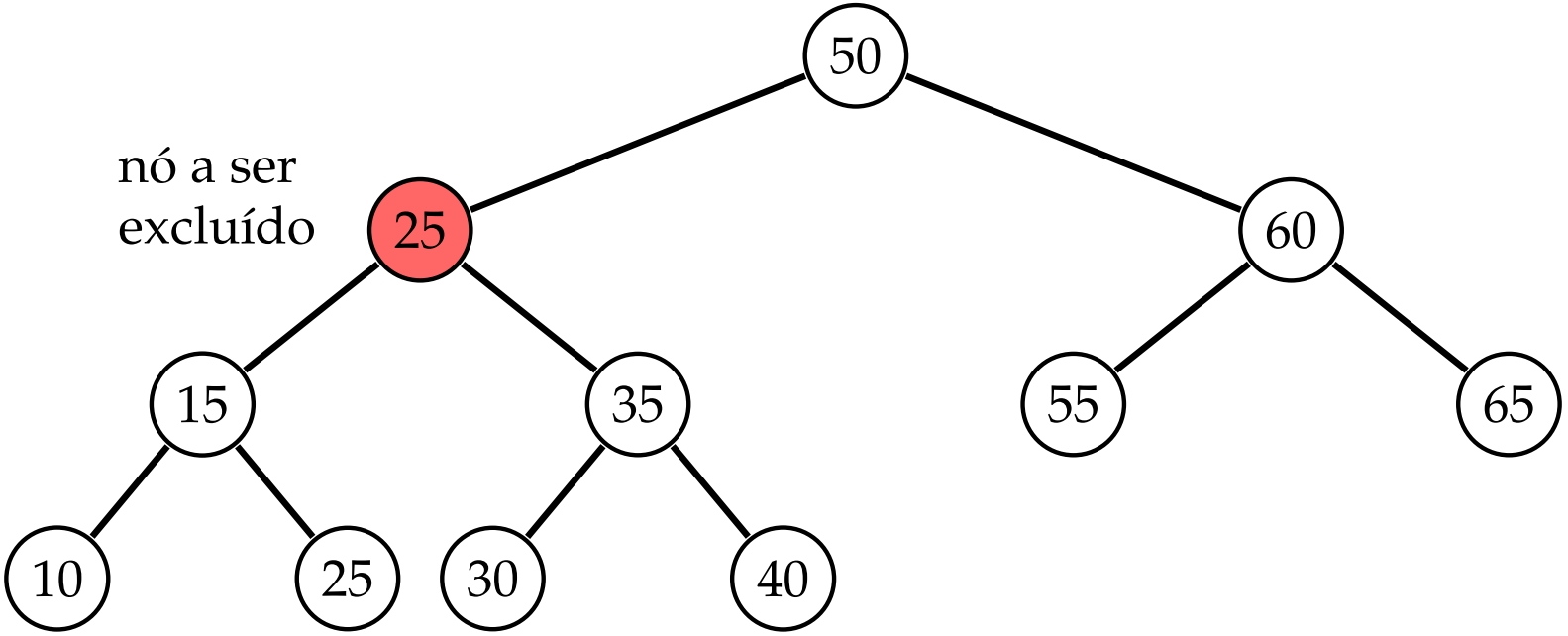
- Para remover um nó com 2 filhos, uma função auxiliar para encontrar o menor elemento da sub-árvore da direita ou substituto (nó mais à esquerda da sub-árvore da direita) será implementada.
- Dado um ponteiro para um nó qualquer `p`, a função `menorSubArvDireita(p)` retorna o menor elemento de sua sub-árvore da direita, se existir.
- Só pode ser chamada para um nó `p` com 2 filhos.

```
No* ArvBinBusca::menorSubArvDireita(No *p)
{
  No *aux = p->getDir();
  while(aux->getEsq() != NULL)
    aux = aux->getEsq();
  return aux;
}
```

Árvore Binária de Busca

Remove nó com 2 filhos

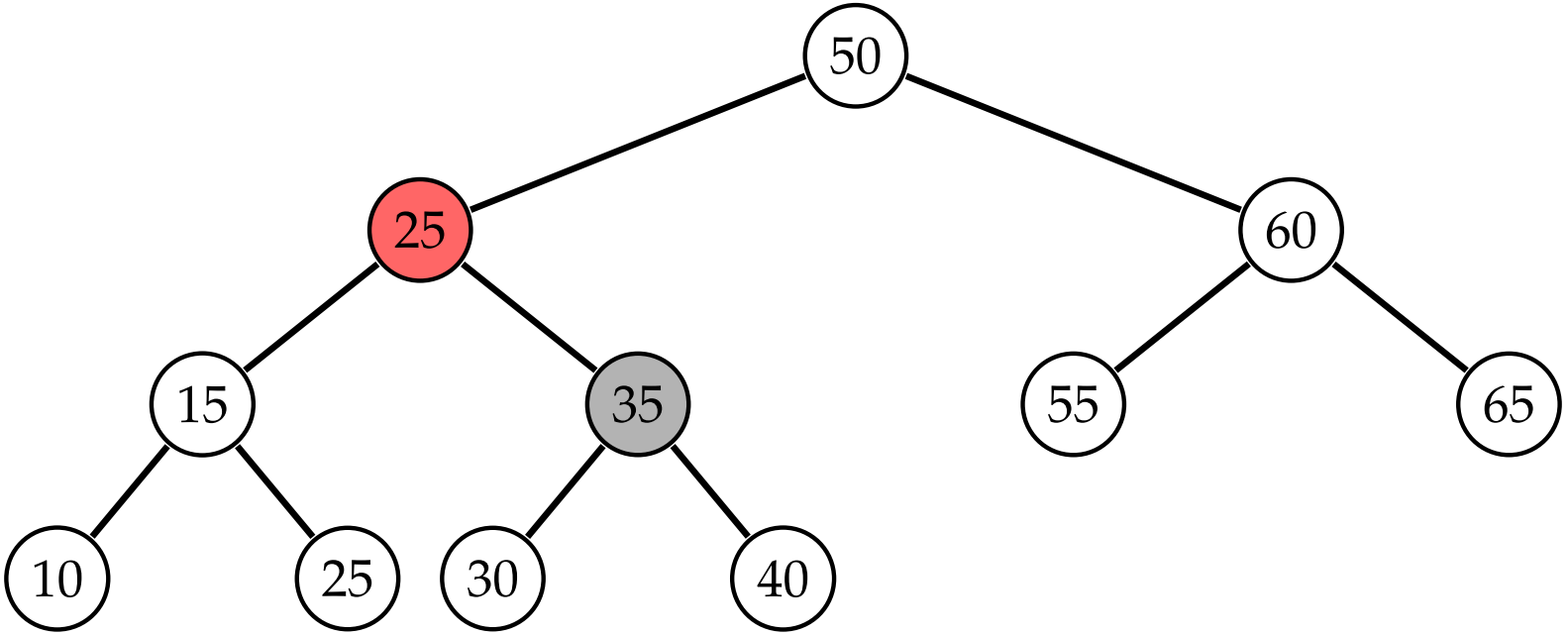
- Exemplo: excluir o nó 25
- Passo 1/6



Árvore Binária de Busca

Remove nó com 2 filhos

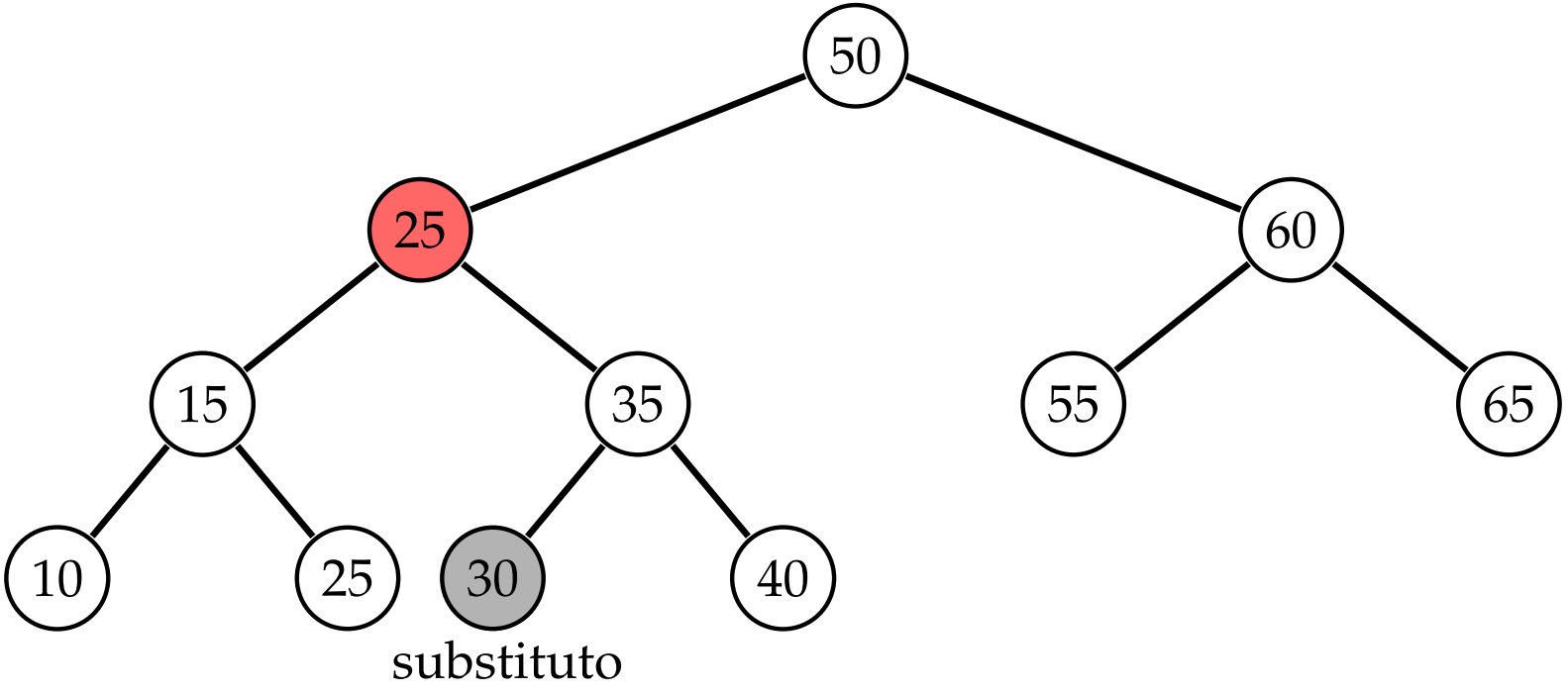
- Exemplo: excluir o nó 25
- Passo 2/6



Árvore Binária de Busca

Remove nó com 2 filhos

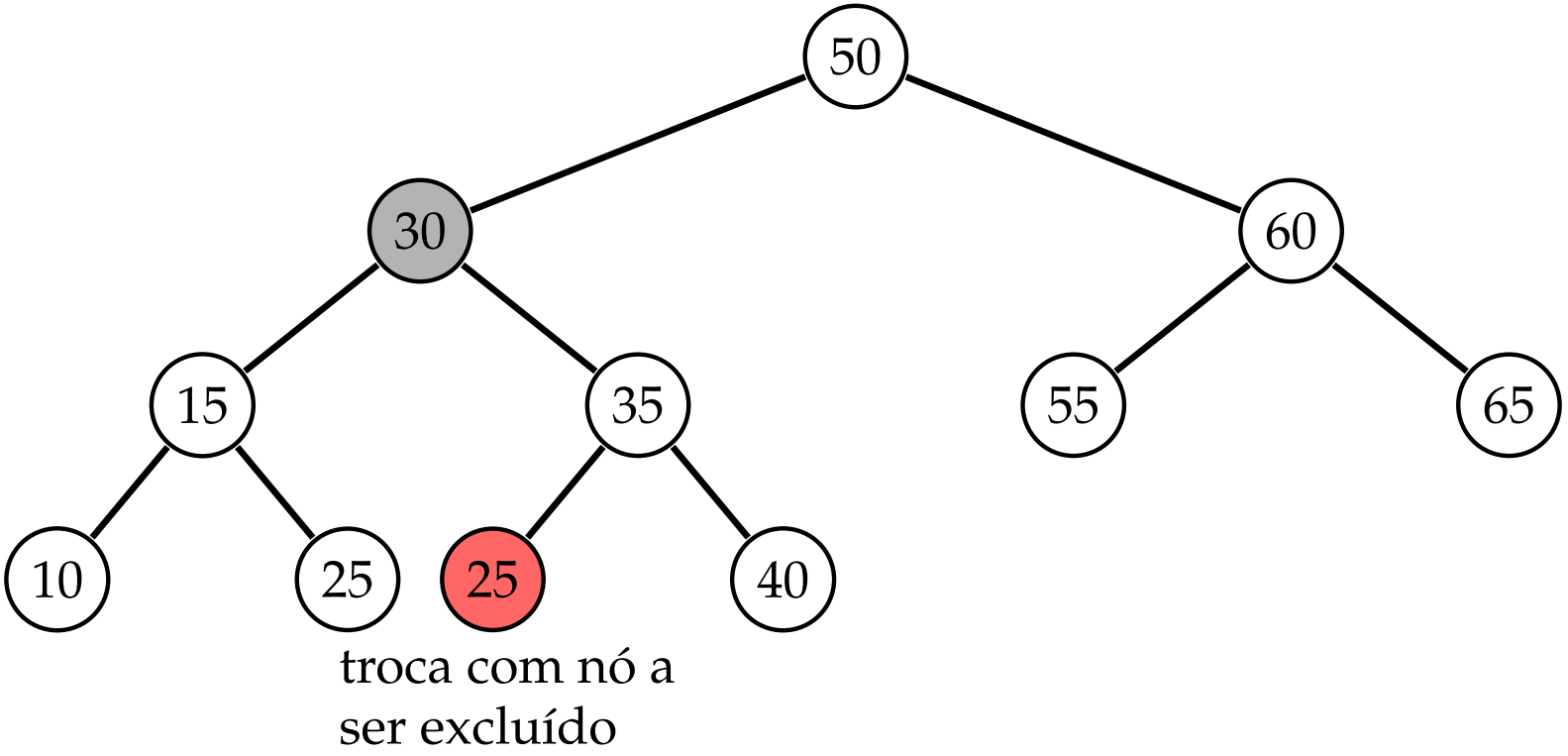
- Exemplo: excluir o nó 25
- Passo 3/6



Árvore Binária de Busca

Remove nó com 2 filhos

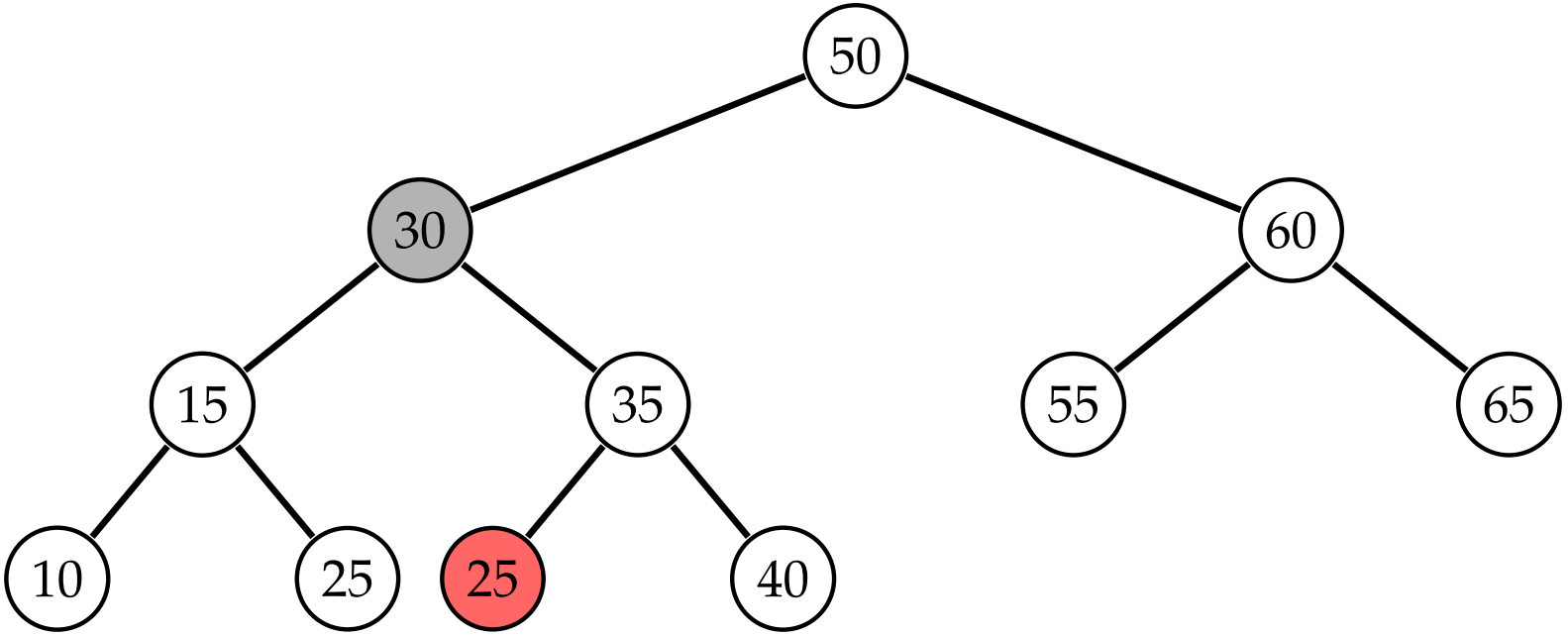
- Exemplo: excluir o nó 25
- Passo 4/6



Árvore Binária de Busca

Remove nó com 2 filhos

- Exemplo: excluir o nó 25
- Passo 5/6

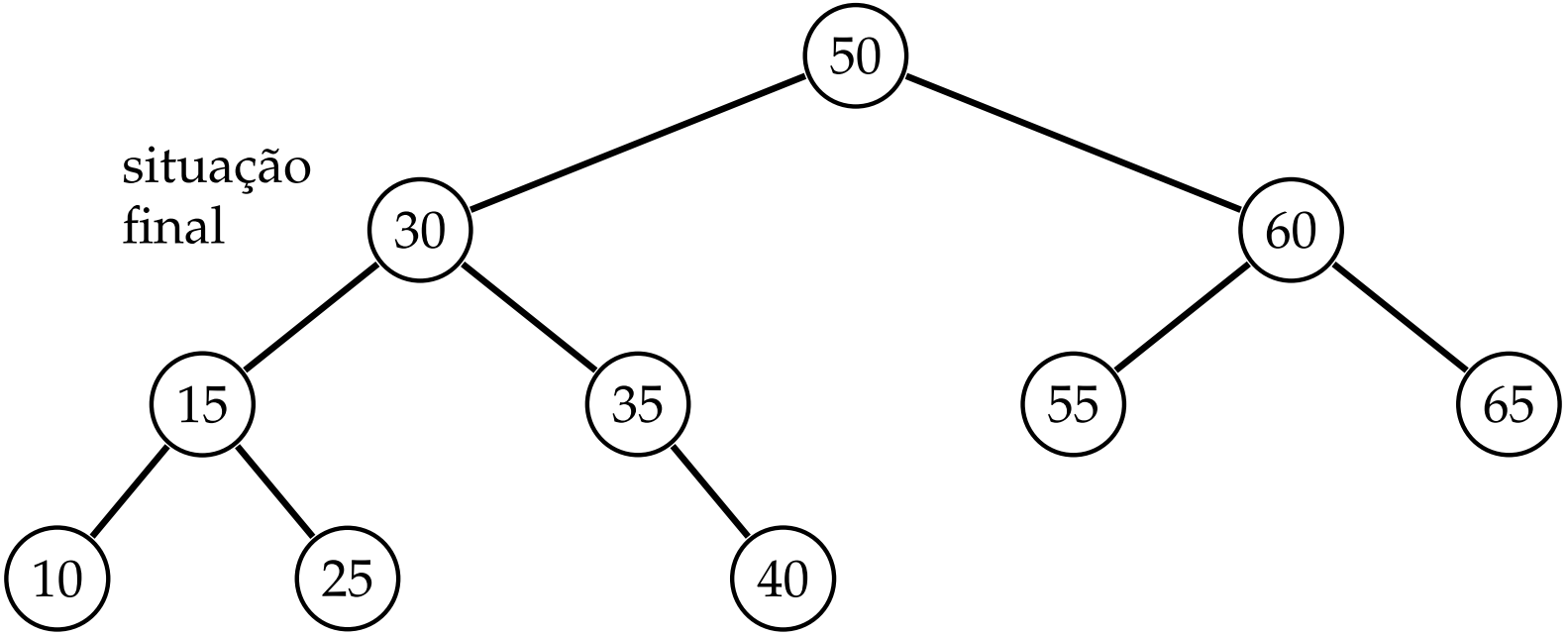


remove nó folha

Árvore Binária de Busca

Remove nó com 2 filhos

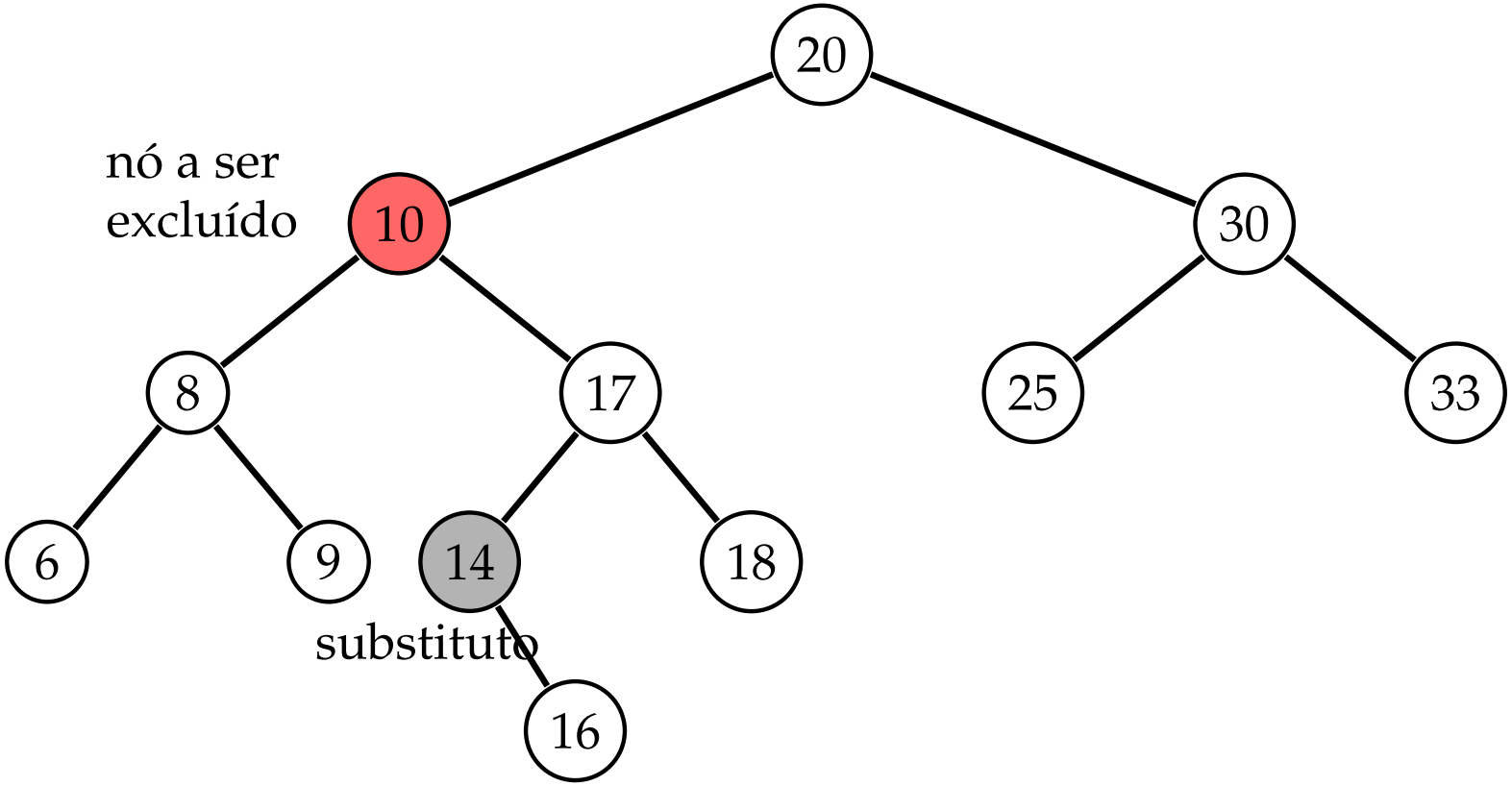
- Exemplo: excluir o nó 25
- Passo 6/6



Árvore Binária de Busca

Remove nó com 2 filhos

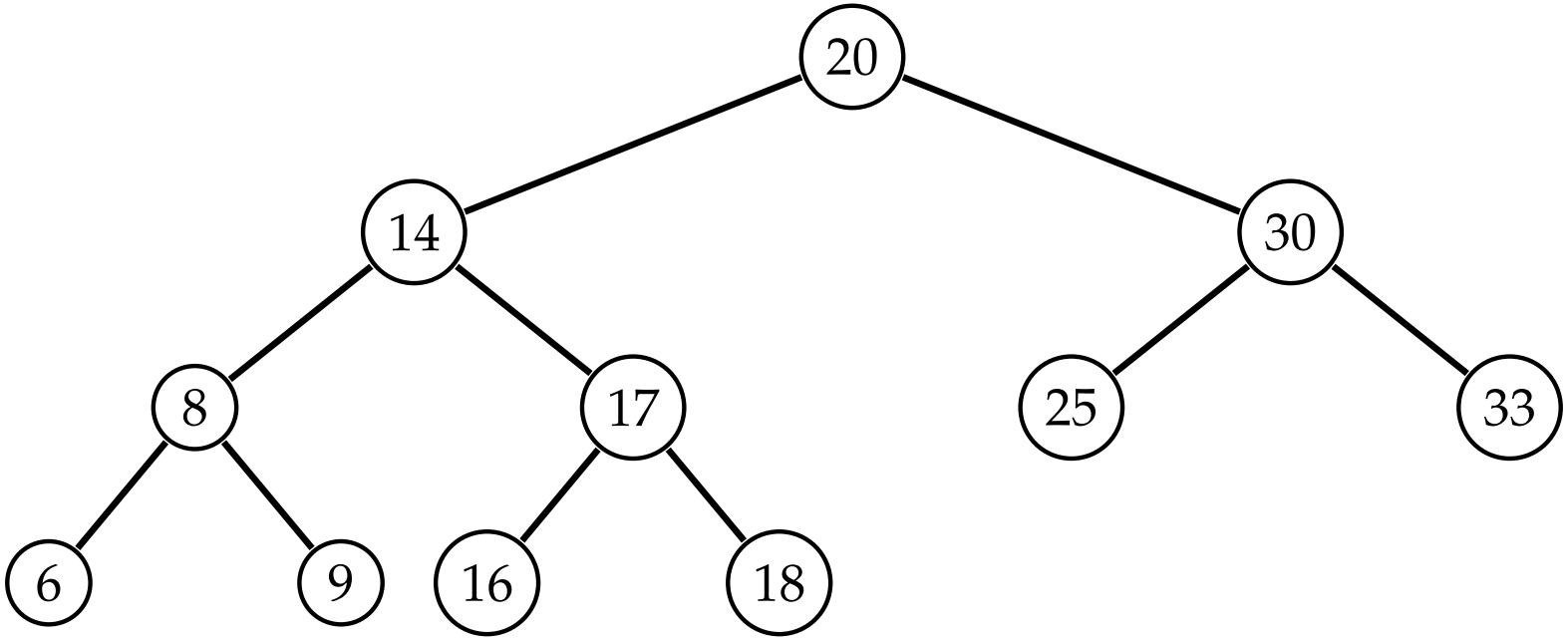
- Outro exemplo: excluir o nó 10



Árvore Binária de Busca

Remove nó com 2 filhos

- Outro exemplo: excluir o nó 10



situação final

Árvore Binária de Busca

Remoção

```
void ArvBinBusca::remove(int x)
{
  raiz = auxRemove(raiz, x);
}
```

- Na classe `ArvBinBusca`

```
class ArvBinBusca {
 private:
   //...
   No* auxRemove(No *p, int x);    // remove no
   No* removeFolha(No *p);         // caso 1
   No* remove1Filho(No *p);        // caso 2
   No* menorSubArvDireita(No *p); // caso 3
   //...
 public:
   //...
   void remove(int x); // chama auxRemove
};
```

Árvore Binária de Busca

Remoção

```
No* ArvBinBusca::auxRemove(No *p, int x)
{
  if(p == NULL)
    return NULL;
  else if(x < p->getInfo())
    p->setEsq( auxRemove(p->getEsq(), x) );
  else if(x > p->consultaInfo())
    p->setDir( auxRemove(p->getDir(), x) );
  else // achou o no a ser removido
  {
    if((p->getEsq() == NULL) &&
       (p->getDir() == NULL))
      p = removeFolha(p);
    else if((p->getEsq() == NULL) ||
            (p->getDir() == NULL))
      p = remove1Filho(p);
    else
    // continua ...
```

Árvore Binária de Busca

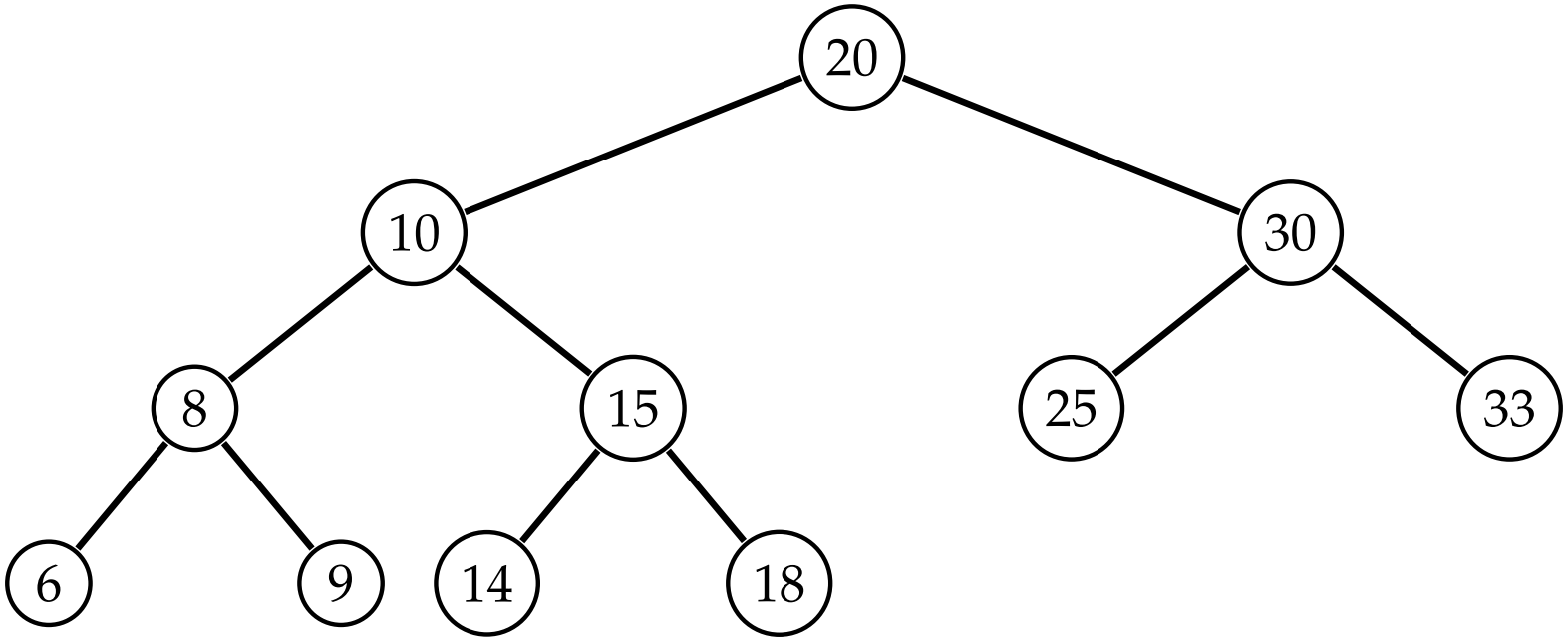
Remoção

```
    // continua ...
    else
    {
      // p tem dois filhos
      No *aux = menorSubArvDireita(p);

      // troca as informacoes
      int tmp = aux->getInfo();
      p->setInfo(tmp);
      tmp->setInfo(C);

      // recursao: para a subarv. direita
      p->setDir( auxRemove(p->getDir(),x) );
    }
  }
  return p;
}
```

Árvore Binária de Busca

Remove nó com 2 filhos

- Exercício
  - Excluir os nós 14, 15, 18 e 6
  - Apresentar a ABB ao final de cada remoção



Exercícios

1. Fazer uma função para encontrar, e retornar, o maior elemento de uma árvore binária de busca.
2. Fazer uma função para encontrar, e retornar, o menor elemento de uma árvore binária de busca.
3. Fazer um procedimento para remover o maior elemento de uma árvore binária de busca.
4. Fazer um procedimento para remover o menor elemento de uma árvore binária de busca.
5. Alterar o(s) procedimento(s) de remoção de nó com dois filhos considerando, agora, o maior elemento da sub-árvore à esquerda como o elemento a ser trocado com o nó a ser removido.

Exercícios

6. Uma árvore binária de busca é considerada balanceada se sua altura h é próxima de $\log_2(n)$. Fazer uma função para verificar se uma dada árvore binária de busca está balanceada. Considere uma árvore balanceada se $h < \log_2(n) + 1$.

7. Fazer mais...

Árvore Binária de Busca

Outras possibilidades

- Utilizar um nó que possui um ponteiro para o pai de um nó.
- Assim, o TAD `No` terá: info, esq, dir, pai